



Convergence enhancement of SIMPLE-like steady-state RANS solvers applied to airfoil and cylinder flows

Antariksh Dicholkar^{*}, Frederik Zahle, Niels N. Sørensen

DTU Wind Energy, Frederiksborgvej 399, Roskilde 4000, Denmark

ARTICLE INFO

Keywords:

Computational fluid dynamics
Reynolds-averaged Navier–Stokes
Steady-state
Aerodynamics
Aerodynamic shape optimization
Wind turbine

ABSTRACT

Iterative steady Reynolds-averaged Navier–Stokes (RANS) solvers are widely used to obtain polars representing the aerodynamic characteristics of airfoils. Unsteadiness is encountered in the flow for many of the simulated angles of attack due to phenomenon such as vortex shedding and separation bubbles. In such cases, the steady RANS solver may become numerically unstable. As a consequence it is unable to find a converged solution, with the solver residuals often entering limit cycle oscillations. The BoostConv method is an alternative to the classical Newton's method to compute unstable steady-state solutions of dynamical systems. It has been previously applied for obtaining steady laminar solutions of the Navier–Stokes equations. However, it demonstrated a lack of robustness by failing to always find the steady-state turbulent solutions of the RANS equations. To overcome this problem, we propose a modification in the application of the BoostConv method to compute steady-state turbulent solutions of the RANS equations. The modified BoostConv method is coupled with an iterative steady RANS solver for computing converged two-dimensional steady-state flows over airfoils. The proposed modification in the application procedure introduces a relaxation parameter that improves the convergence and robustness of the BoostConv method. We show this by finding lift and drag curves with angles of attack spanning 360° for airfoils representative of those used in modern wind turbines. The flow solutions are converged to machine precision at all angles of attack using the novel application procedure of the BoostConv method. Additionally, for flow cases convergent with the original BoostConv method, the modified BoostConv method showed faster convergence rates. The proposed modification provides an enhancement of the BoostConv method that can be easily integrated into existing implementations. By improving the method's robustness and convergence rates in finding steady-state turbulent solutions of the RANS equations, it opens up the modified BoostConv method to be used in a wide variety of industrial flow cases.

1. Introduction

The size of wind turbine rotors have grown considerably over the past four decades, from modest rotor diameters of around 15 m in 1980's to the largest of around 220 m in 2020. The growth of wind turbine rotors is expected to continue into the future fueled by the industry's quest of providing a lower cost of energy. Modern wind turbine blades are up to 90% lighter when compared to blades from the 1980's if they were simply scaled up to match the lengths (Veers et al., 2019). Key aerodynamic innovations such as the use of slender high-lift airfoils in the power producing part of the blade and thick flatback airfoils near the blade root, have facilitated the comparative reduction in blade mass. High-lift airfoils with high relative thickness enable slender and lighter blades. Additionally, the use of airfoils with thick trailing edges (flatbacks) have considerably improved the aerodynamic performance in the structurally vital blade root. Despite

their advantages, airfoils with high relative thickness and thick trailing edges have greater adverse pressure gradients on the upper surface resulting in thicker boundary layers, with a greater chance of unsteady flow phenomena such as flow separation and vortex shedding (Timmer and Bak, 2013).

During the early stages of the wind turbine design process, iterative steady Reynolds-averaged Navier–Stokes (RANS) solvers are widely used to obtain airfoil polars representing the aerodynamic characteristics of airfoils. Steady RANS solvers are also used to perform full wind turbine rotor simulations in the final stages of the design process. But the flow over a rotating wind turbine is inherently unsteady (Sørensen and Michelsen, 2000). As such, steady RANS solvers are unable to accurately capture the flow physics where the flow is separated. On the other hand, time accurate CFD simulations performed using a detached-eddy simulation (DES) model have been shown to provide

^{*} Corresponding author.

E-mail address: acdi@dtu.dk (A. Dicholkar).

<https://doi.org/10.1016/j.jweia.2021.104863>

Received 27 April 2021; Received in revised form 2 November 2021; Accepted 22 November 2021

Available online 20 December 2021

0167-6105/© 2021 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

better accuracy for highly separated flows (Johansen et al., 2002). However, the estimates of the forces obtained from steady RANS solvers provide useful verification of the forces predicted by the engineering models based on the Blade Element Momentum (BEM) theory, that are widely used to design wind turbines. More importantly, steady RANS solvers provide faster simulation times than DES solvers. This translates into faster turnaround times during the design process. Finally, the highly separated flow is mostly encountered in the root and near-root sections of wind turbines under normal operating conditions. These sections of wind turbines are designed with the primary purpose of providing structural support and are not majorly involved in producing power. Thus, the accuracy of simulated flow is not of the highest priority in the root and near-root sections. To some degree the above mentioned factors play a role in the extensive adoption of steady RANS solvers in the wind turbine design process despite their inability of accurately capturing the physics of highly separated flows.

But, steady-state RANS solvers perform well when there is little to no flow separation. When simulating unsteady flow conditions, the solutions of steady-state RANS solvers often do not converge and enter limit cycle oscillations (LCO). Encountering unsteady flow would also make it difficult to perform RANS-based aerodynamic shape optimization. Performing gradient-based steady-state shape optimization of wind turbine blades with a RANS flow solver would prove challenging with partially converged solutions. The gradients of the objective function are computed by linearizing around a converged flow state. For a discrete adjoint method, the linearization ensures that the calculated gradients are the exact gradients of the discrete flow residuals leading to transference of its convergence properties to the adjoint solver (Xu et al., 2015). As a consequence, the non-convergence of the flow solver would cause non-convergence of the adjoint solver leading to inaccurate gradients of the objective function. Since the average of the flow solution undergoing a LCO would not satisfy the steady-state RANS equations, linearization about the averaged LCO would not improve the gradients. Similarly, gradients computed by linearizing about a snapshot of the LCO would be sensitive to the chosen snapshot. Thus, finding a well-converged steady-state flow solution is of paramount importance to gradient computation. Convergence failure can either slow down the optimization process, or prevent the objective function from attaining its optimum. Stable discrete adjoints can be ensured by assured convergence of the steady-state RANS flow solver.

Having converged steady-state flow fields is also useful in performing CFD-based analysis of a more general nature. An example could be the steady RANS-based analysis of highly separated flow over spinnakers, which is a sail designed for sailing off the wind in sailboats in certain situations. Due to highly separated nature of the flow, issues are often encountered in convergence of the steady RANS simulations (Lasher and Sonnenmeier, 2008). Another example could be the analysis of flow over complex terrain. In these cases, the flow is often separated undergoing phenomena such as separation bubbles and bluff body vortex shedding with the RANS residuals entering LCO. At present, either an average of the LCO or a snapshot is taken as the solution on which further analysis is performed. But the average solution does not satisfy the RANS equations and the snapshot is specific to the instant it was captured, leading to uncertainty in the analysis. Even though the unsteadiness in the flow will not be captured due to model limitations, a steady-state solution would satisfy the steady RANS governing equations.

The BoostConv method introduced by Citro et al. in 2017 (Citro et al., 2017a) can be used to accelerate convergence of stable iterative solvers and stabilize non-converging steady-state iterative solvers. This method utilizes Krylov subspaces spanning the residual history to modify the residual at the current iteration step such that the residual at the next step is effectively reduced. Compared to the popularly used Newton-based methods, BoostConv does not involve a Jacobian computation and is thus computationally inexpensive. More importantly, it

only acts on the residual at each iteration allowing for a non-intrusive coupling with an existing iterative solver.

The BoostConv method has been integrated in the open-source high-order spectral element code NEK5000 (Fischer et al., 2008). Since the method is quite recent, only a handful of applications could be found in the literature. All the authors below have used the NEK5000 spectral element code to apply the BoostConv method.

Citro et al. (2015a,b) used the BoostConv method to find the three-dimensional steady-state base flow for investigating the bifurcations formed on inserting a hemispherical roughness element in a laminar Blasius boundary layer. Citro et al. (2016) studied the stability of the wake in the flow behind a sphere rotating in the transverse direction by using the BoostConv method to compute the steady-state base-flow. They tested the method against the classical Newton method to compute the base flow past a circular cylinder and found a difference in the two base flow solutions that was less than 10^{-10} . The computed base flow was also validated against other methods found in the literature. Citro et al. (2017b) used the method to compute the base flow for a global stability analysis on the wake of a three-dimensional stationary sphere with an aim of identifying the second bifurcation that occurs on increasing the Reynolds number beyond the first bifurcation point. Saglietti et al. (2018c,a) utilized the BoostConv method to compute the flow steady states in performing adjoint-based topology optimization of heat sinks in a differentially heated cavity. More recently, Citro (2019) integrated the BoostConv method as a “black-box” with a multigrid solver to accelerate convergence in solving a elliptical Poisson-like equation without additional computational cost. It was noted that applying the method at the finest grid level showed better results than when applied at the coarsest level.

The BoostConv method has been successfully applied to find the steady state of flows over many geometries both in the two-dimensional (2D) and three-dimensional (3D) spaces. However, most of the applications are primarily driven by the goal of finding the steady state base flows required in flow stability analysis. As a result the BoostConv method has been widely used with Navier–Stokes solvers without the inclusion of turbulence models. Only the work of Saglietti et al. (2018c,a) uses the BoostConv method for a purpose other than flow stability analysis. However, here too the BoostConv method has been coupled with a steady-state, incompressible Navier–Stokes solver without inclusion of turbulence models. None of the studies have focused on use cases pertaining to obtaining the steady turbulent flow solutions found by solving the steady-state RANS equations. Furthermore, the efficacy of the method has not been tested for high Reynolds number flows ($Re \gtrsim 10^6$), which are commonly encountered by aircraft and wind turbines.

The application of stabilization methods to obtain steady turbulent RANS solutions are still relatively rare in the literature. The Recursive Projection Method (Shroff and Keller, 1993), the Selective Frequency Damping method (Åkervik et al., 2006), and the Approximate Newton–Krylov (ANK) method (Hicken and Zingg, 2008; Chisholm and Zingg, 2009; Osusky and Zingg, 2013) are few that have been applied to turbulent flow at high Reynolds numbers. Görtz and Möller (2004) coupled the Recursive Projection Method to a non-linear RANS flow solver and applied the method to obtain steady flow for a biconvex airfoil at a Reynolds number of 11×10^6 . Richez et al. (2016) successfully applied the Selective Frequency Damping method to finding the steady-state RANS solutions of turbulent flow around an airfoil at high Reynolds number and near stall. He et al. (2019) analyzed the robustness of the ANK flow solver improved by Yildirim et al. (2019), by performing an aerodynamic shape optimization that started from a circle and resulted in an optimal reference case that was arrived upon separately from a benchmark airfoil. During the optimization the flow solver was able to converge the solution of the RANS equations for infeasible airfoil shapes that violated the equilibrium-flow assumption of RANS and would normally result in invalid results. Madsen et al. (2019) applied the ANK solver (Yildirim et al., 2019) to a successful

3D RANS-based aerodynamic shape optimization of a 10 MW wind turbine rotor with a 178 m diameter. This is considered a challenging case as wind turbine rotors have cylindrical cross-sections near the root section of the blade where the CFD solver is unable to converge the solution of the RANS equations due to violation of the equilibrium-flow assumption. Unlike Madsen et al. (2019), Dhert et al. (2017) did not use the ANK method and had to exclude the root section of the wind turbine blade due to non-convergence of the RANS solution in their optimization.

In this work, we investigate the application of the BoostConv method coupled with a CFD solver for computing converged, 2D steady-state flow over airfoils representative of those used in wind turbine industry. This is facilitated by integrating the method with the 2D version of DTU Wind Energy's in-house incompressible RANS-CFD solver, EllipSys (Michelsen, 1992, 1994; Sørensen, 1995). Its ability to converge non-linear partial differential equations (PDEs) is also verified on simple scalar, one-dimensional and two-dimensional test cases before applying it to airfoils. In relation to the existing literature on this topic, this is the first time that the BoostConv method is being applied to compute steady RANS solutions of high Reynolds number flows. Our goal is to not improve the physics that is being resolved by the RANS model, but to stabilize the numerical RANS solver in situations wherein it is unable to find the converged solutions.

In Section 2 the BoostConv method and the modification in its application procedure is introduced followed by a description of the approach employed in coupling the method with the EllipSys solver. This is followed by a description of the simulation setup including the meshes. The convergence criteria for the simulations is also described. In Appendix the implementation is first validated on simple test cases and a mesh convergence study is presented. In Section 3, the coupled method is applied to cross-sectional geometries representative of a wind turbine blade and their 360° airfoil polars are presented. A parametric study is performed to gain an in-depth understanding of the introduced modifications. In Section 4, the results are discussed with emphasis on adaptations required in the BoostConv implementation to ensure convergence.

2. Methodology

The numerical methods involved in this work consists of the BoostConv formulation coupled to a flow solver. Overviews of the BoostConv method and the EllipSys flow solver are provided in Sections 2.1 and 2.2 respectively. This is followed by a description of the coupling framework implemented to handle the information flow between BoostConv with EllipSys. Changes in the implementation of the BoostConv method to handle parallelization of the flow solver is also described. The modification to the application of the BoostConv method is also presented. In Section 2.4 the numerical setup used for the analysis is presented followed by a description of the computational grid in Section 2.5 followed by the convergence criteria in Section 2.6.

2.1. BoostConv formulation

The central idea behind the BoostConv method is to improve the convergence of an iterative procedure for solving a linear system. The rate of convergence of an iterative solver is dependent on the number of slowly decaying or diverging unstable eigenmodes of the iteration matrix. The BoostConv method seeks to modify the part of the eigenspectrum dictated by the unstable or slowly-decaying modes while the convergent modes are damped by the iterative procedure. It does so by modifying the residual of the iteration at the current step such that the residual at the next step would be reduced. Consider a linear system of equations given by,

$$\mathbf{Ax} = \mathbf{b} \quad (1)$$

where N is the number of equations being solved, $\mathbf{x} \in \mathbb{R}^N$ is the vector of the desired unknowns, $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the matrix of coefficients for the system of equations and $\mathbf{b} \in \mathbb{R}^N$ is the vector of known constant terms. For an iterative method the solution of the system $\mathbf{Ax} = \mathbf{b}$ can be given as

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{B}\mathbf{r}_n, \quad (2)$$

where $\mathbf{B} \in \mathbb{R}^{N \times N}$ is a matrix representing the iterative scheme, n is the current iteration step and $\mathbf{r}_n \in \mathbb{R}^N$ is a vector representing the residual at the current iteration. The residual can be defined as

$$\mathbf{r} = \mathbf{b} - \mathbf{Ax} \quad (3)$$

Citro et al. (2017a) propose a modified residual ξ_n that would replace the residual $\mathbf{r}_n$ in Eq. (2). This modified residual would be a function of the residual $\mathbf{r}_n$ such that using it in Eq. (2) would minimize and ideally nullify the residual in the next iteration $\mathbf{r}_{n+1}$. The expression that is being minimized is shown below,

$$\mathbf{r}_{n+1} = \mathbf{r}_n - \mathbf{AB}\xi_n. \quad (4)$$

Ideally, in order to nullify the residual $\mathbf{r}_{n+1}$ in the next iteration the following relation would have to be satisfied,

$$\mathbf{AB}\xi_n = \mathbf{r}_n. \quad (5)$$

However, the $\mathbf{AB}$ matrix is large, and often ill-conditioned or singular. To obtain the modified residual ξ_n , the solution of Eq. (5) is approximated using a least-squares method by projecting it on a Krylov subspace of vector pairs given by $\mathbf{U} = \text{span}\{\xi_{n-1}\}_1^M$ and $\mathbf{V} = \text{span}\{\mathbf{r}_{n-1} - \mathbf{r}_n\}_1^M$. Here, M is the number of basis vectors. This reduces the size of the problem as $M \ll N$. Taking into account the residual produced by the solution of the least-square problem shown in Eq. (5), the final expression for the modified residual ξ_n becomes

$$\xi_n = \mathbf{Uc} + \mathbf{r}_n - \mathbf{Vc}. \quad (6)$$

where $\mathbf{c} \in \mathbb{R}^M$ is a vector with weights assigned for each basis vector spanning $\mathbf{U}$ and $\mathbf{V}$. The state at the step $n+1$ then can be obtained by applying the modified residual to Eq. (2) by replacing the term $\mathbf{r}_n$ as

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{B}\xi_n. \quad (7)$$

A detailed description of the algorithm with inputs on implementation can be found in Citro et al. (2017a). The BoostConv method can be considered as a predictor–corrector where first the residual vector at the current iteration $\mathbf{r}_n$ is obtained from the iterative method. Then the unstable part of the residual $\mathbf{Vc}$ is subtracted and the least-square correction $\mathbf{Uc}$ is added to the residual. The modified residual ξ_n is then utilized to get the solution at the next step $\mathbf{x}_{n+1}$. On convergence of the iterative procedure, the modified residual ξ_n would effectively be 0 and as such the solution would exactly satisfy the system of equations being solved.

2.2. EllipSys flow solver

The EllipSys flow solver developed by Michelsen (1992, 1994) and Sørensen (1995) is used to carry out the flow simulations in the present work. The solver is capable of performing both two-dimensional and three-dimensional simulations in parallel using the Message Passing Interface (MPI) library. It solves the finite-volume discretization of the incompressible RANS equations on a block-structured grid and in general curvilinear coordinates. The equations are discretized in time using a second-order backward-difference scheme, and in space with a second order central difference scheme for the diffusive terms. In the present work, the convective terms are discretized with the third-order quadratic upwind interpolation for convection kinematics (QUICK) (Leonard, 1979) scheme. The discretized equations are solved using the predictor–corrector approach defined by the Semi-Implicit

Method for Pressure Linked Equations (SIMPLE) (Patankar and Spalding, 1972) algorithm and its variations. The momentum equations are solved at the predictor step using a red/black Gauss–Seidel solver. In the corrector step the pressure-correction equation is solved using a multigrid solver with the additive Schwarz Alternating Method (SAM) enabling the subdomains to be solved simultaneously in parallel. The state variables are stored at the cell centers in a collocated arrangement. This arrangement is known to produce an odd-even pressure decoupling, which is addressed by using the Rhie–Chow interpolation technique (Rhie and Chow, 1983). Menter’s two-equation $k-\omega$ shear stress transport (SST) (Menter, 1993) is used to model the turbulence.

In the present work, the coupled momentum and pressure correction equations are solved using a blend of the SIMPLE and SIM- PLEC (Patankar, 1981) algorithms.

EllipSys is a widely used and well-validated tool in the wind energy research community. It has been used to model 2D flows around airfoils, blade-resolved flows around wind turbine rotors, flows over complex terrain and for wake studies. It has been validated against wind tunnel measurements and compared against the panel code XFOIL (Drela, 1989) for 2D airfoil flows in Bertagnolio et al. (2001), against experimental data for the flow over a wind turbine rotor in the MEXICO project (Bechmann et al., 2011; Sørensen et al., 2016) and for the NREL Phase VI wind turbine (Sørensen et al., 2002; Sørensen and Schreck, 2014). In the context of the present work, the 2D formulation of the EllipSys solver is exclusively used in application to various two-dimensional geometries.

2.3. Coupling BoostConv to EllipSys

The BoostConv algorithm is formulated to work on the residual of a linear iterative system. In the context of the EllipSys CFD-code this would entail applying it first on the residual of the momentum equation which would provide a modified velocity term. This would be followed by its application to the residual of the pressure correction equation which would in-turn provide a modified pressure correction term. However, this approach would be considered intrusive as it would entail modifying the source code of EllipSys.

Alternatively, one iteration of the predictor–corrector approach solving the incompressible RANS system could be considered analogous to an iterative procedure performing a state update. In this case, the residual that would be modified by the BoostConv procedure would then be given as

$$\mathbf{r}_n = \mathbf{w}_{n+1} - \mathbf{w}_n, \quad (8)$$

where $\mathbf{r}_n \in \mathbb{R}^N$ is a vector representing the residual at the current iteration, $\mathbf{w} \in \mathbb{R}^N$ represents a state variable and can be velocity, pressure or the states from the turbulence or transition models in use. This approach has the advantage of being non-intrusive as it only modifies the states obtained at the end of an iteration of the steady RANS solver. This approach is adopted in the present work. It had been first suggested in literature by Bucci (2018), and has been successfully utilized by Saglietti et al. (2018b) in coupling the BoostConv algorithm with the Nek5000 code (Fischer et al., 2008).

2.3.1. Coupling approach

The coupling between the flow solver and the BoostConv method is performed via a wrapper for interacting with EllipSys, called PyEllipsys. This wrapper is coded in the interpreted programming language Python (Van Rossum and Drake, 2009). Since EllipSys is written in the programming language Fortran, C-compliant interfaces are required to be implemented into the flow solver to allow interaction with an external wrapper. The interoperability with C is facilitated in Fortran by the `bind(C)` attribute and the intrinsic module `iso_c_binding` (Gfortran team, 2021). This allows the routines in the flow solver to be called by another C-interoperable programming language, which in this case is the Python wrapper PyEllipsys. Through this interface PyEllipsys can

control the entire execution sequence of EllipSys by allowing the user to interact with the compiled code during runtime. It can initialize the flowfield variables and constants, trigger various solver algorithms and physical models, carry out the solution over an iteration step and write output files. Through various set and get methods the interface allows data transfer with PyEllipsys during runtime. In order to be accessed by PyEllipsys the flow solver is compiled as an external library in the form of a Linux shared object.

The BoostConv method is implemented in pure Python as an auxiliary model in PyEllipsys. At the end of one iteration step of the predictor–corrector algorithm in EllipSys the difference between the new state variables and their values before the iteration are retrieved by PyEllipsys using the “get methods”. For a particular state variable, this difference is considered as the residual that forms the input to the BoostConv method. The modified residual obtained from the BoostConv method is added to the old value of the state variable, altering the value of the state at the end of the iteration. Using a “set method” the updated state variables are transferred back to EllipSys. The modified state variables become the input for the next iteration. (See Fig. 1.)

2.3.2. Implementation

EllipSys works with block structured meshes wherein the computational domain is divided into multiple sub-domains. Information is exchanged across the boundaries of these sub-domains through a single ghost-cell layer surrounding each one of them. Domain decomposition also allows for the flow problem to be solved on multiple processors in parallel using the MPI library. The number of computational sub-domains are divided equally among the number of computer processor units provided to solve the problem. At a minimum, one block would be distributed per processor. This occurs when the number of processors are equal to the number of blocks. Whereas at the other extreme all computational blocks could be assigned to a single available processor.

The BoostConv algorithm of Citro et al. (2017a) has been implemented in a manner that leverages domain decomposition of EllipSys. In BoostConv, two Krylov subspaces are constructed with one spanning the modified residual $\mathbf{U} = \xi_{n-1}$ and the other spanning the difference between consecutive unmodified residuals from the solver $\mathbf{V} = \mathbf{r}_{n-1} - \mathbf{r}_n$. Up to M basis vectors going back M iterations from the current iteration n are stored. The coefficient vector $\mathbf{c}$ shown in Eq. (6), is obtained by solving a least-squares system

$$\mathbf{D}\mathbf{c} = \mathbf{t} \quad (9)$$

where $\mathbf{D} \in \mathbb{R}^{M \times M}$ is a symmetric matrix, $\mathbf{c} \in \mathbb{R}^M$ is the unknown coefficient vector and $\mathbf{t} \in \mathbb{R}^M$ is known. The known terms are computed as,

$$\mathbf{D} = \begin{bmatrix} \mathbf{V}_1 \cdot \mathbf{V}_1 & \mathbf{V}_1 \cdot \mathbf{V}_2 & \cdots & \mathbf{V}_1 \cdot \mathbf{V}_M \\ \mathbf{V}_2 \cdot \mathbf{V}_1 & \mathbf{V}_2 \cdot \mathbf{V}_2 & \cdots & \mathbf{V}_2 \cdot \mathbf{V}_M \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{V}_M \cdot \mathbf{V}_1 & \mathbf{V}_M \cdot \mathbf{V}_2 & \cdots & \mathbf{V}_M \cdot \mathbf{V}_M \end{bmatrix} \quad (10)$$

and

$$\mathbf{t} = \begin{bmatrix} \mathbf{V}_1 \cdot \mathbf{r}_n \\ \mathbf{V}_2 \cdot \mathbf{r}_n \\ \vdots \\ \mathbf{V}_{M-1} \cdot \mathbf{r}_n \\ \mathbf{V}_M \cdot \mathbf{r}_n \end{bmatrix}. \quad (11)$$

Each term of the basis $\mathbf{V}$ is associated with a cell center in the domain, allowing for different contiguous parts of the vector $\mathbf{V}$ to be composed independently in the sub-domains. As seen in Eq. (10) the elements of matrix $\mathbf{D}$ are dot products of the various historical snapshots of the vector $\mathbf{V}$. As such, the element of $\mathbf{V}$ stored at a cell center would either be multiplied by itself or its historical counterpart. As a result, a fraction of every element of the matrix $\mathbf{D}$ is computed independently in a sub-domain. It is important to note that there might be several computational sub-domains assigned to a single processor. As discussed

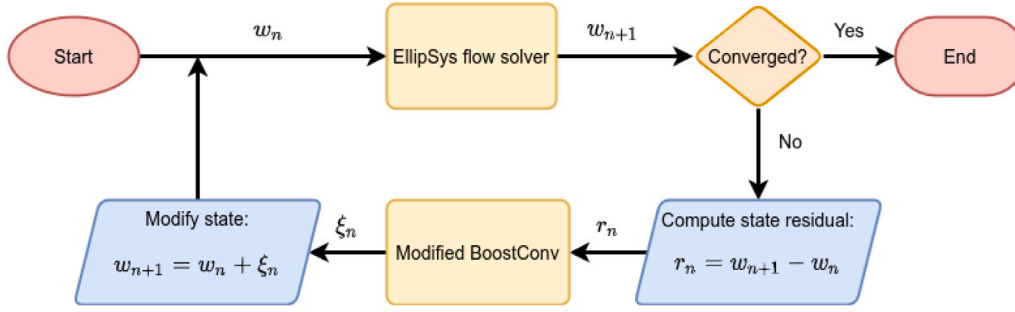


Fig. 1. Workflow depicting integration of the modified BoostConv method with the EllipSys flow solver.

earlier, this value depends entirely on the number of processors made available for the computation. So even though the computations being discussed are sub-domain independent, they might not be processor independent. The independently computed parts of the elements of the matrix $\mathbf{D}$ are stored in a vector form and are transmitted to each of the allocated processors using `MPI_Allgather` from the MPI library. On each processor the component-wise sum of the received vectors provides the elements of the matrix $\mathbf{D}$. The vector $\mathbf{t}$ is computed in a similar manner. Each allocated processor then solves the least-squares system shown in Eq. (9).

The adopted procedure of making every processor solve the least-square problem should be regarded as a proof of concept and not the final form of the implementation. It operates under the assumption that compared to the computational cost of solving the flow, the least square problem is much smaller in size and would not be a major contributor to the computational cost. We have not spent effort in parallelizing the solution to the least-square problem and comparing its performance with the current procedure. In the current procedure, the least-square problem constitutes a serial part of the code. As the size of the least-square problem is proportional to the chosen basis size, larger basis sizes would incur greater computational costs. As a consequence, the procedure will not scale well with basis size. Parallel implementation of the least-square problem is required and will be tackled in the future.

In the current implementation, computational savings are gained by leveraging the symmetry of the matrix $\mathbf{D}$ by computing $M \times (M + 1)/2$ elements instead of the full matrix. There are two distinct phases involved in building the Krylov subspaces. In the first phase, for every new iteration of the flow solver the basis vectors $\mathbf{V}$ and $\mathbf{U}$ are stored in memory. The basis dimensions are increased continually until the maximum user-specified basis size of M . Beyond this, for every new iteration only the oldest basis is discarded and the latest basis is added to the stored memory space. Thus, the memory size involved in storing the Krylov subspaces does not change beyond the first $M + 1$ iterations of the flow solver. This property of the algorithm translates into a reduction in the number of dot product operations required per iteration to M .

2.3.3. Novel application procedure for the BoostConv method

Bucci (2018) highlights difficulties in applying the BoostConv algorithm to the Navier–Stokes system due to the ill-conditioned correlation matrix $\mathbf{D}$. This is addressed by skipping a fixed number of flow solver iterations between two residual snapshots instead of using successive iterations for the residual snapshots required to construct the Krylov subspace. It is recommended that the number of skipped iterations between the two residual snapshots should neither be too low or too high. The BoostConv method is devised for a linear system. Capturing successive residual snapshots in a non-linear system leads to a noisy modified residual output and may not result in convergence. Whereas, skipping flow solver iterations in sampling successive residuals does not capture some of the non-linearities present in the solution. On the other hand, skipping too many flow solver iterations in sampling the

residuals can entirely disregard the non-linear nature of the Navier–Stokes system, leading to non-convergence of the solution. Recall that without the BoostConv method, the flow solver would provide the solution state shown here as

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{B}\mathbf{r}_n, \quad (12)$$

where $\mathbf{B} \in \mathbb{R}^{N \times N}$ is a matrix representing the iterative scheme, n is the current iteration step and $\mathbf{r}_n \in \mathbb{R}^N$ is a vector representing the residual at the current iteration. The BoostConv method proposes a modified residual ξ_n that would replace the residual $\mathbf{r}_n$ in Eq. (12). The new equation reads as

$$\mathbf{x}_{n+1}^* = \mathbf{x}_n + \mathbf{B}\xi_n, \quad (13)$$

where $\mathbf{x}_{n+1}^*$ denotes the state produced on application of the BoostConv method. Note that $\mathbf{x}_{n+1}$ is the state produced by the flow solver without the application of the BoostConv method. The modified residual ξ_n is given as,

$$\xi_n = \mathbf{r}_n + \mathbf{U}\mathbf{c} - \mathbf{V}\mathbf{c}. \quad (14)$$

where $\mathbf{c} \in \mathbb{R}^M$ is a vector with weights assigned for each basis vector spanning $\mathbf{U}$ and $\mathbf{V}$. On substituting the expression for ξ_n in Eq. (13), we obtain the expression for the new state $\mathbf{x}_{n+1}^*$ as,

$$\mathbf{x}_{n+1}^* = \mathbf{x}_{n+1} + \mathbf{B}(\mathbf{U}\mathbf{c} - \mathbf{V}\mathbf{c}). \quad (15)$$

In the present work, a relaxation parameter is introduced in the BoostConv algorithm to facilitate its application to a non-linear system. This parameter α is multiplied with the term modifying the residual from the solver in Eq. (15), resulting in

$$\xi_n = \mathbf{r}_n + \alpha(\mathbf{U}\mathbf{c} - \mathbf{V}\mathbf{c}). \quad (16)$$

which when rewritten in the form of Eq. (15) becomes

$$\mathbf{x}_{n+1}^* = \mathbf{x}_{n+1} + \alpha(\mathbf{U}\mathbf{c} - \mathbf{V}\mathbf{c}), \quad (17)$$

where $\alpha \in [0, 1]$. For $\alpha = 0$ the flow solver operation without any contribution from the modifying residual is recovered. Whereas, for $\alpha = 1$ the original BoostConv procedure is recovered.

The relaxation parameter α can be continuously varied between 0 and 1 unlike the solution presented in Bucci (2018). Such a property is desired when used in an aerodynamic shape optimization framework, which is the intended goal for the developments done in the present work.

In their paper introducing the BoostConv method, Citro et al. (2017a) make a provision for inclusion of such a relaxation parameter in the term $\mathbf{B}$ that represents the iterative scheme, seen here in Eq. (15). Prior to the present work, the documented applications of the BoostConv method set this term to unity and did not include a special realization of this term. We propose a linear realization of this term $\mathbf{B}$ via the relaxation parameter α .

An interpretation of the introduced relaxation parameter α can be obtained by viewing Eq. (17) as a gradient descent problem. Then the residual modification term $\mathbf{U}\mathbf{c} - \mathbf{V}\mathbf{c}$ obtained from the BoostConv

Table 1

Flow settings.

Density	1.0 kgm ⁻³
Relative velocity	1.0 ms ⁻¹
Inflow k	0.01 m ² s ⁻²
Inflow ω	10 ⁶ s ⁻¹
Dynamic viscosity μ	1/Re kgm ⁻¹ s ⁻¹

algorithm can be regarded as the step direction in the minimization problem and the introduced relaxation parameter α as the step size.

It should be noted that in the remainder of this paper the application of the BoostConv method with the inclusion of the relaxation parameter α that is not unity is referred to as the “modified BoostConv method”. Whereas its application with α set to unity is referred to as the “original BoostConv method”.

2.4. Simulation setup

Three cross-sectional geometries representative of a modern wind turbine blade are simulated in the present work. The geometries include a cylindrical cross-section representing the root region of the wind turbine blade. The power-producing region of the blade is represented by the NACA 63(3)-418 (Bertagnolio et al., 2001) airfoil with a relative thickness of 18%. An airfoil with a high-relative thickness and a blunt trailing edge, also known as a flatback airfoil, represents the near-root region. This airfoil cross-section is generated by increasing the relative thickness of the FFA-W3-360 (Bertagnolio et al., 2001) airfoil to 50% and the trailing edge thickness to 30% of the chord length. The chord length for all the geometries are set to unity.

All the simulations aim to achieve the steady-state solution of the incompressible RANS equations. The QUICK scheme is used to discretize the convective terms along with a version of the SIMPLEC pressure correction scheme of Kolmogorov et al. (2015). Turbulence is modeled with the $k-\omega$ SST scheme of Menter (1993). Here k refers to the turbulent kinetic energy and ω to the specific dissipation rate. To remain consistent and simulate potential aerodynamic shape optimization conditions, the same pressure and velocity under-relaxation values for the SIMPLEC scheme are used for all the simulations. Similarly, the same inflow and far-field turbulence settings are used across the simulations.

The simulations are performed by setting the flow density and incoming flow velocity to unity. Since the characteristic length of the geometries are given by their chord lengths and are also set to unity, the Reynolds number (Re) of the flow is varied by changing the flow viscosity. The settings kept constant for the simulations are shown in Table 1.

The flow over the NACA 63(3)-418 airfoil and the flatback airfoil are simulated at a Reynolds number of 1×10^7 . Whereas, the cylindrical cross-section is simulated for Reynolds numbers ranging from 1×10^2 to 1×10^7 .

2.5. Computational mesh

The block structured grids are generated using the 2D hyperbolic grid generator HypGrid2D (Sørensen, 1998). For all three geometries, an O-mesh is generated with 384 cells distributed around the surface and 192 cells in the normal direction. The O-mesh for the cylinder can be seen in Fig. 2(a). This mesh is the finest mesh used in the simulations and is called the L1 mesh. A coarser mesh with 192 cells distributed around the surface and 96 cells in the normal direction, called the L2 mesh, is obtained by removing every second cell from the L1 mesh in the two directions. Finally, the coarsest mesh, called the L3 mesh, is obtained by repeating the cell removal process for the L2 mesh. The L3 mesh has 96 cells distributed around the surface and 48 cells in the normal direction.

Fig. 2(b) shows the zoomed in profile of the cylinder and the mesh near the cylinder wall shown at grid level 2 (L2 mesh) for improved visualization. Similarly, the profiles and the near wall meshes at grid level 2 are shown for the NACA 63(3)-418 airfoil and the 50% thick flatback airfoil in Figs. 3(a) and 3(b), respectively. A first cell height of 1.5×10^{-6} corresponding to a $y^+ \leq 1$ for the simulated flow Reynolds numbers (Re) is considered. The outer mesh boundary is approximately 45 chord lengths away from the surface. The geometries are modeled as no-slip surfaces. The mesh boundary is specified as a velocity inlet except in the region downstream of the geometry covering an azimuth of ± 45 degrees around the incoming flow direction. The mesh boundary in this region is specified as a flow outlet wherein it is assumed that the flow is fully developed with the normal gradients at the outlet boundaries being enforced as zero. The mesh is divided into 18 blocks of 64×64 cells with a total of 73728 cells.

Large pressure gradients exist at the trailing edge of an airfoil especially at high angles of attack. A pointed trailing edge causes highly skewed cells in this sensitive region, negatively affecting the convergence of the simulation. This is prevented by slightly opening up the trailing edge. For the NACA 63(3)-418, the trailing edge is opened up to have a thickness measuring 0.1% of the chord length. However, the flow solution is sensitive to the sharp corners resulting from the opened trailing edge for higher order convective schemes. This is in spite of being better than the pointed trailing edge on multiple mesh quality parameters such as cell smoothness, skewness and expansion ratio. The mesh quality in the trailing edge region is further improved by smoothing the sharp corners using a Laplacian smoother. This distributed the cells in a manner that prevented high cell expansion rates and skewed faces. The smoothed trailing edges for both the NACA 63(3)-418 airfoil and the 50% thick flatback airfoil in Figs. 4(a) and 4(b), respectively.

2.6. Convergence criteria

The solution from the EllipSys flow solver is considered to be converged when the ratio of the L1 norm of the residuals at a given iteration is below a specified tolerance. These residuals are taken from the momentum equations for the flow in the X and Y cartesian directions, the pressure correction equation and the $k-\omega$ SST turbulence equations. The L1 norm for each of the residuals is computed by summing up the magnitude of the respective discrete residuals computed throughout the mesh. The L1 norms are then normalized by the L1 norm of the residual after the first iteration. The solution is considered converged if the maximum of this ratio is less than or equal to the specified tolerance.

For the simulations performed in this study, the tolerance for convergence is set a 10^{-12} . Thus, the maximum of the ratios of the L1 norms has to be less than 10^{-12} for the simulation to be considered converged. For the initial conditions set for the simulated flow cases, it is seen that the residual L1 norm ratios would achieve values slightly below 10^{-12} before ceasing further decrease. At this point, the solutions are considered to be converged at or near machine precision. In finding the L1 norm, we compute the sum of the absolute values of the residuals over all the mesh cells. For the finest mesh level (L1) being simulated, the total number of cells amount to 73728. If on an average the residual in each mesh cell is to obtain the machine precision absolute value of 10^{-16} then the effective L1 norm would be approximately 7×10^{-12} . Thus, the near machine precision convergence criteria of 10^{-12} is justified. The convergence criteria can be represented by the relation,

$$\frac{\|R^n\|_1}{\|R^1\|_1} \leq 10^{-12} \quad (18)$$

where R^n represents the residuals after the n th iteration and R^1 the residuals after the first iteration.

It is important to note that the residuals being described here are not to be confused with the residuals used with the BoostConv method described in Eq. (8). The residuals being used for the BoostConv method are based on the difference in the state variable such as velocity and pressure, over an outer iteration of the EllipSys solver.

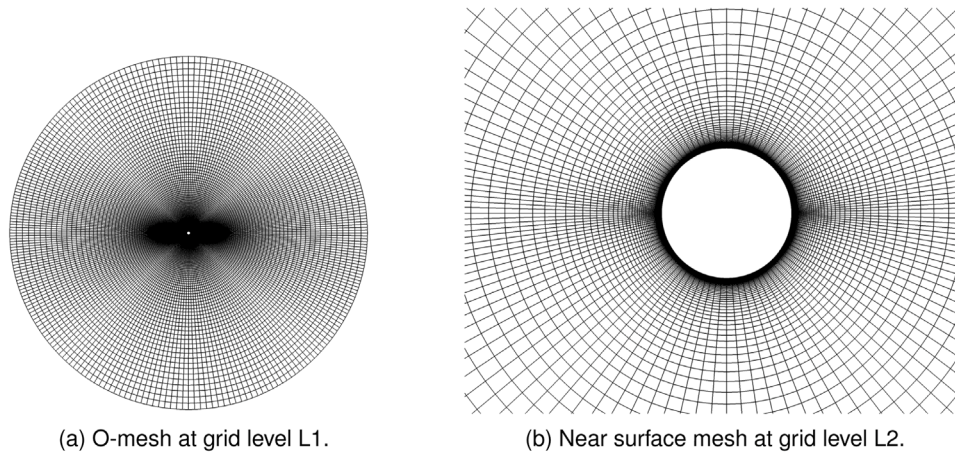


Fig. 2. Computational mesh for computing the RANS solution over a 2D cross-section of a cylinder.

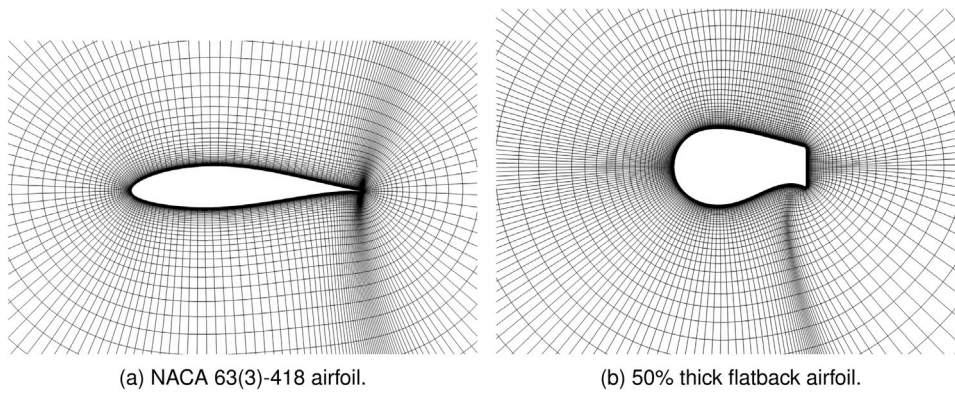


Fig. 3. Near surface computational mesh and airfoil geometry for the NACA 63(3)-418 airfoil and the flatback airfoil at grid level L2.

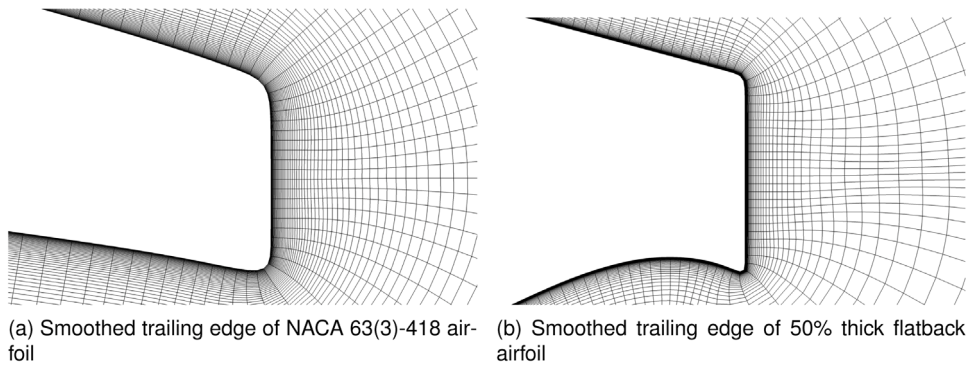


Fig. 4. Near surface computational mesh at grid level L3 shown for the smoothed trailing edges of the NACA 63(3)-418 airfoil and the flatback airfoil.

3. Results

3.1. Computational cost

The computational cost imposed by the current implementation of the modified BoostConv method on a RANS simulation carried out by the flow solver is shown in Table 2. The NACA 63(3)-418 airfoil is chosen for the simulation. The airfoil is simulated for 1000 outer iterations of the EllipSys flow solver. As described in Section 2.5, the computational domain for the NACA 63(3)-418 airfoil is decomposed into 18 blocks of 64×64 mesh cells. Computational resources are allocated such that each computing processor is distributed one block. We choose the fastest possible configuration for the EllipSys flow solver to gauge the worst possible impact of the current implementation of

the modified BoostConv method. As described in Section 2.3.2, in its current implementation every allocated processor is made to solve the least-square problem constructed in the modified BoostConv method. Additionally, the size of the least-square problem scales with the chosen basis size.

As shown in the left most column of Table 2, the basis size is varied from 0 to 160. A basis size of 0 corresponds to the standard flow solver with no contribution from the modified BoostConv method. The central column of Table 2 shows the total computational time spent per iteration for each basis size. The right most column of Table 2 shows the contribution of the current implementation of the modified BoostConv method as a percentage of the total computational time of the simulation. It is seen that the current implementation of the modified BoostConv method has an acceptable computational cost until

Table 2

Computational cost of the modified BoostConv method for varying basis sizes as a percentage of the total computation time.

Basis size	Wall time per iteration [s]	Time spent on the BoostConv method [%]
0	0.022	0.0
5	0.026	16.27
10	0.029	23.82
20	0.031	30.49
40	0.035	38.12
80	0.045	51.25
160	0.084	74.04

a basis size of 40, where 60% of the computational time is spent on the flow solver and 40% on the modified BoostConv method. Even though acceptable, a 40% computational time cannot be considered negligible. For a basis size of 80, equal amounts of time is being spent on the modified BoostConv method and on solving the flow. Beyond a basis size of 80 the contribution of the modified BoostConv method dominates the total computational time.

The current implementation of the modified BoostConv method operates under the assumption that the size of the least square problem is much smaller than the flow being solved. This is seen to hold until a basis size of 40. For the modified BoostConv method to be competitive for larger basis sizes the least square problem has to be solved in a parallelized manner. Finally, it is important to note that the computational times seen here are only valid when each block has been assigned to a single computing processor. In many large flow problems this is not always possible. As such the computational cost of the modified BoostConv method shown here is the worst that can be achieved for the current implementation.

3.2. Cylinder

Fully turbulent flow over a 2D cross-section of a cylinder is simulated over a range of Reynolds Numbers from 1×10^2 to 1×10^7 . Specifically, these are given below:

$$Re = \{10^2, 5 \times 10^2, 10^3, 5 \times 10^3, 10^4, 2 \times 10^4, 5 \times 10^4, 10^5, 2 \times 10^5, 5 \times 10^5, 10^6, 5 \times 10^6, 10^7\} \quad (19)$$

The simulations are performed based on the turbulence settings for the $k-\omega$ SST turbulence model shown in Table 1. The drag and lift coefficients resulting from the simulations are shown in Figs. 5(a) and 5(b), respectively. In these figures, the black markers (x) represent the solution obtained from the standard solver which here is the EllipSys solver without the BoostConv method. Whereas, the solid green line represents the solution obtained from the solver with the modified BoostConv method. In the figures, the pink highlighted region depicts the Reynolds numbers for which the standard RANS solver did not converge and showcased limit cycles. For all Reynolds numbers outside the pink highlighted region, the standard solver converged to machine precision and obtained the same values as that from the boosted solver. For the Reynolds numbers lying inside the pink highlighted region, the drag and lift coefficients of the simulations from the standard solver are obtained by averaging the solution. In Fig. 5(a), the averaged drag values obtained from the standard solver are seen to be higher than the deeply converged values from the boosted solver. Similarly, in Fig. 5(b) the averaged lift coefficient values from the standard solver are seen not to be exactly 0 as is expected from a deeply converged solution. The pink highlighted region is bound by the Reynolds numbers 5×10^2 and 5×10^4 . In Fig. 5(a) the start of the pink highlighted region is marked by the departure of the black markers (x) representing the standard EllipSys solver, from being coincident with the green solid line representing the boosted EllipSys solver, at a Reynolds number of 5×10^2 . The end of the pink highlighted region occurs at a Reynolds number of 5×10^4 . Beyond this the standard EllipSys solver is once again

convergent. This is indicated in Fig. 5(a) when the black markers (x) representing the standard EllipSys solver begins to coincide with the green solid line of the boosted EllipSys solver at a Reynolds number of 1×10^5 .

3.3. Airfoils

The 360° lift and drag profiles are obtained for the NACA 63(3)-418 airfoil and the 50% thick flatback airfoil at a Reynolds number of 1×10^7 . The shape of the NACA 63(3)-418 airfoil and the flatback airfoil can be seen in Figs. 3(a) and 3(b), respectively. The simulations are performed for angles of attack ranging from -176° to $+180^\circ$ at an interval of 4° . The resulting drag and lift profiles are shown in Figs. 6(a) and 6(b) for the NACA 63(3)-418 airfoil, and in Figs. 7(a) and 7(b) for the flatback airfoil.

In the figures, the solid green line represents the EllipSys solver with the modified BoostConv method. The black dashed line represents the standard EllipSys solver. Both the boosted and standard solvers are executed until their solutions converge to machine precision. Converged solutions are obtained with the boosted solver at all angles of attack. Whereas, for a significant number of angles of attack the solutions of the standard solver do not converge and display limit cycle oscillations. These regions of non-convergence of the standard solver have been depicted by the pink highlighted regions. In these regions, the lift and drag values for the standard solver are obtained by averaging the solution over the iterations for which they undergo the limit cycle. There are two distinct regions of non-convergence of the standard EllipSys solver seen for the NACA 63(3)-418 airfoil. These regions, highlighted in pink in Figs. 6(a) and 6(b), include the angles of attack ranging from -144° to -56° , and from 48° to 140° . For the flatback airfoil there are six pink highlighted regions observed, as seen in Figs. 7(a) and 7(b). These include the angle of attack at -144° , and the angles of attack ranging from -160° to -152° , from -132° to -120° , from -84° to -68° , from -4° to 12° and from 76° to 96° . In Figs. 6(a) and 7(a) the drag coefficients obtained from the boosted solver are seen to be lower than the averaged values of the standard solver in the pink highlighted region. Differences between the two are also observed in the lift coefficient values in the pink highlighted region seen in Figs. 6(b) and 7(b).

3.4. Effect of changing the BoostConv relaxation parameter

The behavior of the relaxation parameter α introduced in the modified BoostConv algorithm shown in Eq. (17) is studied. This is achieved by varying the value of the BoostConv relaxation parameter for a fixed basis size on selected simulation cases. For each case, the flow parameters such as Reynolds number, angle of attack of the incoming flow, the settings for the $k-\omega$ SST turbulence model, the mesh, and the velocity and pressure relaxation values of the SIMPLE-like algorithm are kept constant.

Three simulation cases are chosen such that they do not converge with the standard EllipSys flow solver and result in a limit cycle. These include the 2D cross-section of the cylinder, the NACA 63(3)-418 airfoil and the 50% thick flatback airfoil. The relevant flow settings for the three geometries are shown in Table 3. The simulations were carried out on the fine L1 O-meshes for 1×10^6 iterations of the EllipSys flow solver. The meshes have been described in Section 2.5. The inflow and far-field settings for the turbulence model are also kept the same as that described in Section 2.4.

A maximum basis-size of 60 is chosen in the modified BoostConv method for the two airfoil geometries whereas a maximum basis-size of 15 is chosen for the cylinder. On completion of the basis building phase of the method, the modified residual at each step is computed based on sampling of the residual space at previous iterations. The residual space window encompasses the previous number of iterations equal to the chosen maximum basis-size. In all the simulations of the

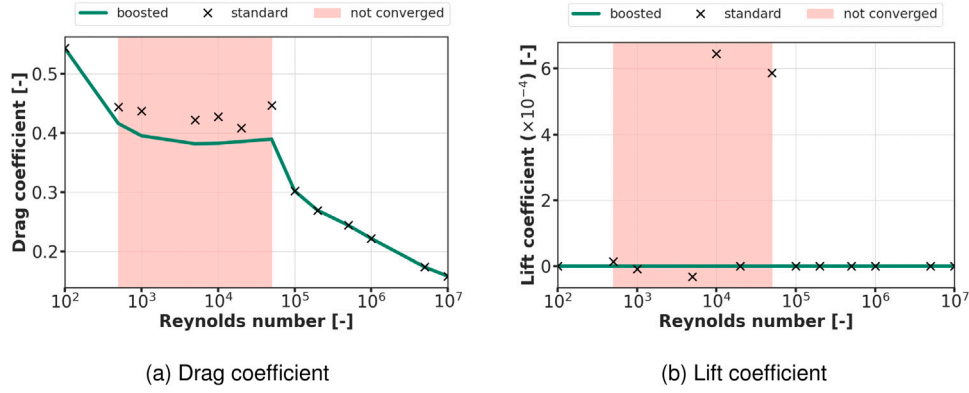


Fig. 5. Lift and drag coefficients for flow over a 2-D cross-section of a cylinder over Reynolds numbers ranging from 1×10^2 to 1×10^7 .

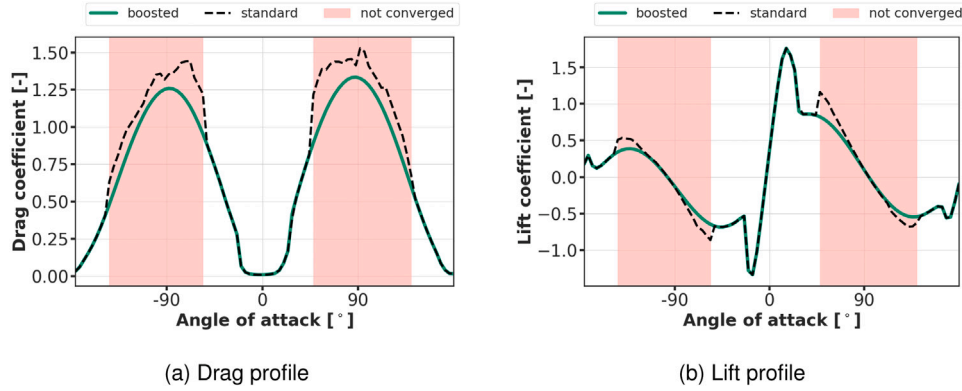


Fig. 6. 360° lift and drag profiles for NACA 63(3)-418 at a Reynolds number of 1×10^7 .

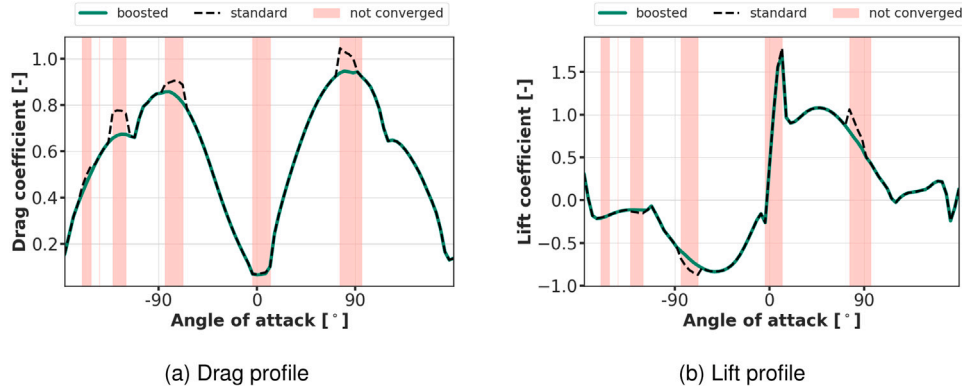


Fig. 7. 360° lift and drag profiles for the thick flatback airfoil at a Reynolds number of 1×10^7 . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 3

Flow settings.

Geometry	Reynolds number	Angle of attack
Cylinder	2×10^4	0°
NACA 63(3)-418 airfoil	10×10^6	52°
Flatback airfoil	10×10^6	88°

parametric study, the modified BoostConv method is activated after running the standard solver for the first 3000 iterations. It takes around 3000 iterations for the solution of the standard solver to develop and enter into a limit cycle. For the purpose of the parametric study, the relaxation parameter α is varied from a value of 0.001 to a value of 1. To recall, a value of $\alpha = 1$ recovers the original BoostConv method

of Citro et al. (2017a). Whereas a value of $\alpha = 0$ does not modify the residual and recovers the standard flow solver. The main intention of modifying the BoostConv method with the relaxation parameter α is to account for the non-linear behavior of the RANS flow solver.

The effect of the BoostConv relaxation parameter α on the convergence behavior of the flow solver for the 2D cylinder cross-section can be seen in Fig. 8(b). The convergence behavior of the EllipSys flow solver is represented by the residual for the velocity in the positive X-direction (U-velocity residual). For an $\alpha = 1$ it is seen that the flow is not convergent and enters a limit cycle. But a reduction in the value of the relaxation parameter α makes the flow solver convergent. Fig. 8(b) shows that the flow solver is convergent for $\alpha = 0.6, 0.4, 0.1, 0.05$. It then again enters a limit cycle for the values $\alpha = 0.01, 0.005, 0.001$. This indicates the presence of a valley like region for the relaxation

Table 4
Standard settings for the SIMPLE under-relaxation factors.

Geometry	Velocity under-relaxation	Pressure under-relaxation
Cylinder	0.8	0.9
NACA 63(3)-418 airfoil	0.8	0.4
Flatback airfoil	0.8	0.4

parameter α wherein the modified BoostConv method makes the flow solver convergent. Additionally, the convergence rate seems to be increasing with decreasing values of α within the convergent valley-like region. This cannot be concretely established from Fig. 8(b) as the trend is broken by $\alpha = 0.1$ which shows an abrupt decrease in the convergence rate, indicating the presence of slow decaying eigenmodes. In the parameter study shown in Fig. 8(b) there might be a sub-region in the parameter space around $\alpha = 0.1$ wherein the convergence rate gradually decreases as one moves from $\alpha = 0.4$ to $\alpha = 0.1$ before increasing once again. However, the presence of such a sub-region cannot be ascertained by the coarse resolution of α in the current parameter study.

The valley-shaped convergence region is also clearly observed in Figs. 9(b) and 10(b). Fig. 9(b) shows the U-velocity residuals for various values of α for the NACA 63(3)-418 airfoil simulated at a Reynolds number of 10 million with the flow incident at an angle of attack of 52° . Similarly, Fig. 10(b) shows the same for the 50% thick flatback airfoil simulated at a Reynolds number of 10 million with the flow incident at an angle of attack of 88° . In Fig. 9(b) for NACA 63(3)-418 airfoil, the convergent region is also seen to include the case for $\alpha = 1.0$. At $\alpha = 1.0$ the original BoostConv method is recovered. Fig. 9(b) demonstrates that it is possible for the original BoostConv method to converge some of the flow cases seen in this paper. However, the flow cases that are convergent with the original BoostConv method ($\alpha = 1.0$) are rarely observed, occurring at some unique combinations of angle of attack, Reynolds number and under-relaxation settings for the SIMPLE algorithm. The introduction of the relaxation parameter α increases the robustness of the original BoostConv algorithm allowing for convergent behavior over a variety of flow cases.

For the flatback airfoil shown in Fig. 10(b), the residual is not convergent for the original BoostConv method at $\alpha = 1.0$. On reduction of α the residual becomes convergent. The valley-like convergent region is once again seen for the α parameter space. The residual convergence rate increases with a decreasing value of α until a value of $\alpha = 0.01$. For $\alpha = 0.005, 0.001$ the residual is not convergent and enters a limit cycle.

3.5. Effect of changing the SIMPLE under-relaxation factors

The pressure and velocity under-relaxation factors have a significant impact on the convergence rate of the SIMPLE-like flow solver. Furthermore, it has been seen from the results presented until now that the BoostConv relaxation parameter α is necessary to obtain a deeply converged steady-state for flow cases when the solutions from a RANS solver enter a limit cycle. The value of α is also seen to affect the residual convergence rate. The inter-dependency of the SIMPLE under-relaxation factors and the BoostConv relaxation parameter is explored because of their influence on the solver convergence. The parametric studies discussed until now shown in Figs. 8(b), 9(b) and 10(b) were performed with the SIMPLE under-relaxation factors set at values that would be considered standard for the simulated cases. The standard under-relaxation settings are shown in Table 4. The same parametric studies are repeated with more conservative values for the SIMPLE under-relaxation factors as shown in Table 5.

Fig. 8(a) shows the parametric study for the cylinder case performed for conservative values of the SIMPLE under-relaxation factors. Compared to the study with standard values of the SIMPLE under-relaxation factors shown in Fig. 8(b), the conservative case does not converge for the values $\alpha = 0.1, 0.05$. Additionally, an overall slowdown

Table 5
Conservative settings for the SIMPLE under-relaxation factors.

Geometry	Velocity under-relaxation	Pressure under-relaxation
Cylinder	0.6	0.2
NACA 63(3)-418 airfoil	0.6	0.2
Flatback airfoil	0.6	0.2

of the convergence rates are observed in the conservative case. This is exemplified by the residual at $\alpha = 0.01$ taking almost twice the number of iterations to converge in the conservative case than in the standard case. A difference in the convergent region for the α parameter space is also seen for $\alpha = 0.005$. At this value, the conservative case is convergent whereas the standard case is not.

Changes in the convergence rates and in the convergent regions for the α parameter spaces are also seen for the NACA 63(3)-418 and flatback airfoils. For the NACA 63(3)-418 airfoil, the residual is no longer convergent for the original BoostConv method ($\alpha = 1.0$) in the conservative case as seen in Fig. 9(a). Whereas, the residual at $\alpha = 1.0$ is convergent for the standard case seen in Fig. 9(b). The residual with the best convergence rate for both conservative and standard cases occurs at $\alpha = 0.01$. However, this residual converges in the conservative case for almost twice the number of iterations of the standard case. For the flatback airfoil, the residuals at $\alpha = 0.4$ and $\alpha = 0.6$ are not convergent for the conservative case, as seen in Fig. 10(a). On the other hand, the residuals at the same values of α are convergent for the standard case shown in Fig. 10(b). Conversely, the residual at $\alpha = 0.005$ is non-convergent for the standard case but becomes convergent for the conservative case. Finally, similar to the cylinder and NACA 63(3)-418 airfoil cases, the best convergence rate for both the conservative and standard cases is seen at $\alpha = 0.01$. The number of iterations in which this residual converges in the conservative case is almost twice of the standard case.

4. Discussion

The standard EllipSys RANS solver can sometimes fail to find converged steady-state solutions where the unsteadiness in the flow due to phenomena such as vortex shedding or laminar separation bubbles, gets resolved by the mesh. Instead, in such cases, the solution displays limit cycle oscillations.

The standard EllipSys flow solutions for the cylindrical cross-section exhibited non-convergent behavior between the Reynolds numbers of 5×10^2 and 5×10^4 . As described by Singh and Mittal (2005) in their numerical study of flow past a cylinder cross-section, the flow in this region is unsteady and is marked by vortex shedding which starts about a Reynolds number of 5×10^2 . The presence of numerical instabilities makes it difficult for the standard EllipSys RANS solver to converge the highly separated unsteady flow being resolved in this region. The solution fails to converge and enters limit cycle oscillations. On the other hand, in the same Reynolds number range, the EllipSys solver coupled to the modified BoostConv method is able to find the converged unstable steady-state solutions. The trend of the drag values observed between the Reynolds numbers of 5×10^2 and 5×10^4 exhibits a similarity between the trend seen for the cylinder “drag-crisis” which occurs between 5×10^4 and 2×10^5 . However, any correlation is purely speculative since we perform fully turbulent simulations without the inclusion of a transition model. Without a transition model the flow physics resulting in the cylinder “drag-crisis” should not be captured.

The standard EllipSys RANS solver also fails to converge for configurations involving high degrees of flow separation of the 18% thick NACA 63(3)-418 airfoil and the 50% thick flatback airfoil. Whereas, the EllipSys solver coupled to the modified BoostConv method finds the converged steady-state solutions at all the simulated angles of attack for both the NACA 63(3)-418 airfoil and the flatback airfoil. At high angles of attack beyond the static stall angle, the NACA 63(3)-418

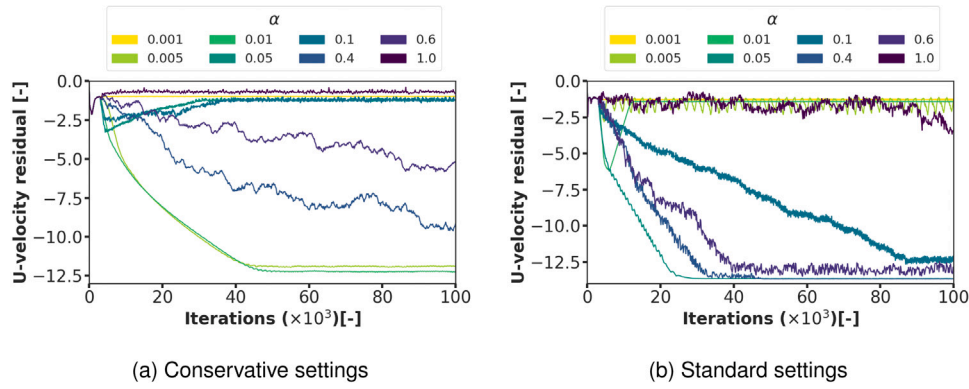


Fig. 8. Parametric study for cylinder at a Reynolds number of 2×10^4 for conservative and standard values of the SIMPLE pressure and velocity under-relaxation factors.

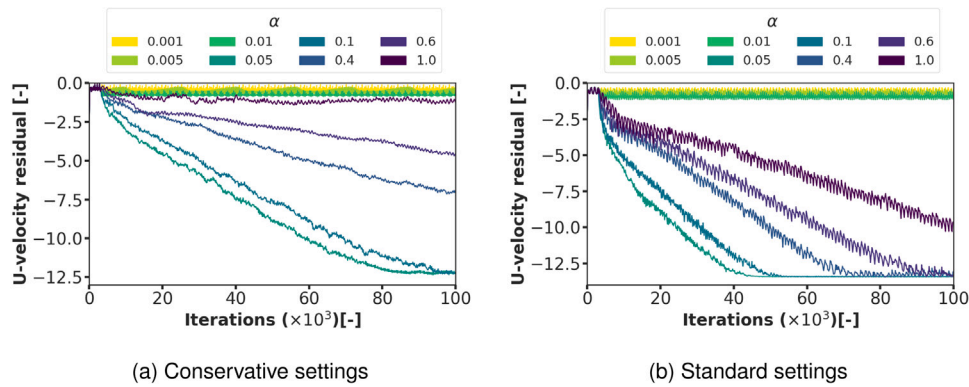


Fig. 9. Parametric study for NACA 63(3)418 airfoil for at angle of attack 52° for conservative and standard values of the SIMPLE pressure and velocity under-relaxation factors.

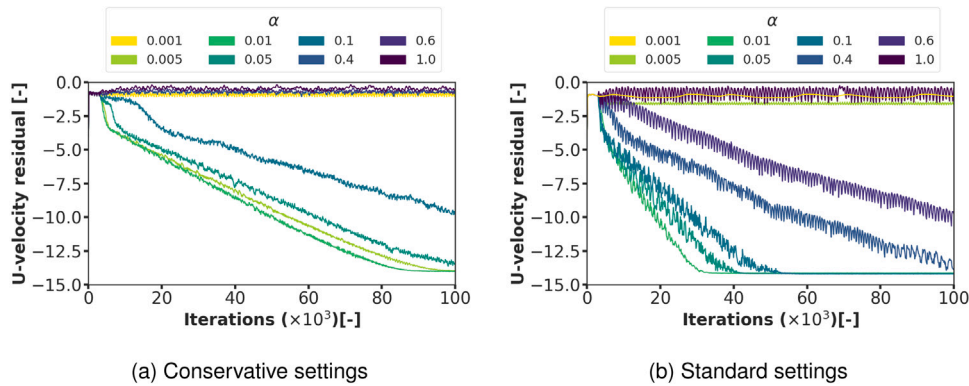


Fig. 10. Parametric study for the 50% thick flatback airfoil for at angle of attack 88° for conservative and standard values of the SIMPLE pressure and velocity under-relaxation factors.

airfoil enters the deep stall region. At this stage the flow separation point occurs at the leading edge with the flow being characterized by highly unsteady phenomena such as vortex shedding (Timmer and Bak, 2013). In the deep stall region, the standard EllipSys RANS solver is not able to find the converged steady-state and the solution exhibits limit cycle oscillations. For the NACA 63(3)-418 airfoil, the behavior of the flow in deep stall explains the presence of two distinct regions beyond the positive and negative static stall angles where the standard EllipSys solver fails to converge. However, the flow over the 50% thick flatback airfoil behaves differently to the relatively thin NACA 63(3)-418 airfoil. For the flatback airfoil, the standard EllipSys solver fails to converge for the low angles of attack between the negative and positive static stall

angles. This can be attributed to the shape of the flatback airfoil. With a relative thickness of 50% and a relatively higher nose radius than most conventional airfoils, the flatback airfoil undergoes bluff body separation at low angles of attack. The contribution of the resulting pressure drag can be noted by the significantly higher drag coefficient for the flatback airfoil at low angles of attack when compared to the more streamlined NACA 63(3)-418 airfoil.

The lack of robustness of the original BoostConv method in applications to 2D airfoil flows is shown by setting the modified BoostConv relaxation parameter to unity. Simulated flow cases that are convergent with the original BoostConv method ($\alpha = 1.0$) are rarely observed, occurring at some unique combinations of angle of attack, Reynolds

number and under-relaxation settings for the SIMPLE algorithm. Such a unique case is chosen for the NACA 63(3)-418 airfoil wherein the residuals are convergent for the original BoostConv method. Despite being convergent the original BoostConv method is seen to be the slowest in convergence when compared to using lower values of α . The introduction of the modified BoostConv relaxation parameter α increases the robustness of the original BoostConv algorithm allowing for convergent behavior over a variety of flow cases. The parameter α offers a “valley-like” convergent region between the values of 0 and 1. This convergent region contains a sub-region of α values for which the observed convergence rates are the highest. From the simulated cases it is seen that the highest convergence rates occur for α between 0.01 and 0.05. The “valley-like” convergent region can be explained by viewing the application of the BoostConv algorithm, shown in Eq. (17), as a gradient descent problem. The residual correction offered by the BoostConv algorithm can then be regarded as the direction of descent, and the parameter α becomes the step size. The flow solver residual can be considered as the cost function of the residual minimization problem. If the step size is too large the minimization problem tends to not converge, whereas for a step size that is too small the minimization problem would converge in a considerably large number of iterations. The “valley-like” convergent region is the result of selecting step-sizes that are appropriate or optimal for the problem. Instead of a fixed step size used in this study, the convergence could be further improved by using line-searching algorithms such as exact line search or a backtracking line search (Boyd et al., 2004).

For a simulation made convergent by the modified BoostConv method, the value of the SIMPLE under-relaxation factors affects both the residual convergence rates and the size of the “valley-like” convergent region. Conservative SIMPLE under-relaxation settings result in a reduction of convergence rates for all convergent values of α compared to the convergence rates for more aggressive under-relaxation factors. A reduction of 0.2 in both the velocity and pressure under-relaxation factors halved the convergence rates, for the same value of α . A change in the under-relaxation factors also directly affected the convergent region for the α parameter space. However, it could not be determined whether the size of the convergent region expanded or reduced with a change in the under-relaxation factors. This is because the conservative under-relaxation factors opened up the convergent region in the lower end of the α parameter space for $\alpha = 0.005$ in the cylinder and flatback airfoil. On the other hand, they reduced the convergent region in the higher end of the α parameter space, for the residuals at $\alpha = 0.4, 0.6$ for the flatback airfoil, and at $\alpha = 1.0$ for the NACA 63(3)-418 airfoil.

The introduction of the relaxation parameter α in the BoostConv method greatly improves the robustness of the method by accounting for the non-linear nature of the underlying RANS equations. For solutions convergent with the original BoostConv method, the relaxation α allowed for improvement of the convergence rates. But the presented results and any ensuing generalizations are limited by the RANS solver and the selected application cases. The chosen steady-state RANS solver is incompressible and is based on a SIMPLE-like algorithm. A compressible RANS solver might behave differently with the BoostConv method due to the difference in the formulation of the solved RANS equations.

The incompressible, steady-state EllipSys RANS solver uses a SIMPLE-like algorithm with the momentum and pressure equations being solved in a segregated manner using a fixed point iteration. Whereas, in a monolithic approach the RANS system of equations is solved as a whole using a Newton-based method. Such a coupled approach is considered to be more robust than the segregated one (Heil et al., 2008). The present study demonstrating the effectiveness and behavior of the modified BoostConv method is limited by its application to a RANS solver based on a segregated approach. The modified BoostConv method's behavior might differ for a monolithic approach.

Despite being generally applicable, our enhancement of the BoostConv method has so far only been applied with the flow solver described in this work and has not been tested with other flow solvers.

In this context, it should be noted that the performance of the original BoostConv method (Citro et al., 2017a) is limited by the flow solver, applied turbulence models and flow cases presented in this study. The efficacy of the original BoostConv method has been shown in various applications, both 2D and 3D, simulated using the NEK 5000 spectral element code. The applications have tackled 3D flow around spheres (Citro et al., 2015a,b, 2016, 2017b), in a cubic cavity (des Bosc and Kuhlmann, 2019) and through a 90°-bend pipe (Lupi et al., 2020). In the 2D domain, the method has been used to study flow around circular cylindrical cross-sections (Lemke et al., 2021; Negi et al., 2020; Sierra et al., 2020; Boujo et al., 2019) and in cavities (Saglietti et al., 2018c,a). Additionally, all the applications involved the stabilization of flow solutions computed using either Direct Numerical Simulations (DNS) of the Navier–Stokes equations without the inclusion of a turbulence model, or for laminar flows.

However, prior to the present study there is no evidence in the literature of the original BoostConv method being applied to a RANS solver with a turbulence model. Neither is there any evidence of the BoostConv method being applied to high Reynolds number flow over airfoils. It can be speculated that the lack of robustness of the original BoostConv method is confined to SIMPLE-like steady state RANS solvers with a $k - \omega$ SST turbulence model. If this is indeed the case, then the modification to the BoostConv method introduced in this study might not be required with other types of flow solvers and flow conditions. Regardless, the modification does allow both the original BoostConv method and the nascent solver to be recovered. Thus, the modification should be regarded as an enhancement of the original BoostConv method and not a replacement.

5. Conclusion

In this paper, the application of the BoostConv method to find the steady state turbulent solutions of the RANS equations for high Reynolds number flow ($Re \approx 10^6$) is studied. For this purpose, the BoostConv method is coupled to a segregated flow solver based on the SIMPLE algorithm, called EllipSys. The incompressible, steady RANS equations are solved for 2D flows over cross-sectional geometries representative of modern day wind turbines. These include, a 18% thick airfoil representing the power producing part, a 50% thick flatback airfoil representing the near-root section and a cylindrical cross-section representing the root section of the blade.

The BoostConv method demonstrated a lack of robustness by failing to always find the steady state RANS solutions of the simulated cases. It is postulated that the failure of the BoostConv method stems from applying a method derived for linear systems to the highly non-linear RANS equations. A modification to the BoostConv method is proposed that accounts for the non-linearities. The modification involves the introduction of a parameter α that relaxes the residual correction. The relaxation parameter can take a minimum value of 0 and a maximum value of 1. At $\alpha = 1$, the original BoostConv method is recovered. Whereas at $\alpha = 0$, the flow solver behavior without the BoostConv method is recovered. This property positions the proposed modification as an enhancement of the original BoostConv method. When the residual minimization offered by the BoostConv algorithm is viewed as a gradient descent problem, the introduced relaxation parameter α can be regarded as the step size responsible for scaling the step direction in the form of the residual correction, provided by the algorithm.

The modification considerably improves the convergence and robustness of the BoostConv method. We show this by using the modified BoostConv method to find the fully turbulent 360° lift and drag polars for both the 18% thick airfoil and the 50% thick flatback airfoil. The RANS solutions in the simulated flow cases are converged up to machine precision at all angles of attack. Additionally, we find the converged steady state solutions for the cylindrical cross-sections at Reynolds numbers ranging from 1×10^2 to 1×10^7 .

A parametric study is performed to explore the effect of the parameter α on the convergence behavior of the flow residuals. A valley-shaped region is identified in the parameter space wherein the flow residuals are convergent. Even for flow cases that are convergent with the original BoostConv method, decreasing the α showed faster convergence rates. The under-relaxation factors of the flow solver are found to influence the size of the valley-shaped convergent region while also affecting the convergence rates. From the simulated flow cases, it is noted that highest convergence rates occur for α between 0.01 and 0.05. Based on the evidence we recommend that the values in this range be regarded as the best first choice for simulations carried out with the modified BoostConv method. It is further suggested that instead of using a fixed parameter α , line searching algorithms such as exact line search and backtracking line search can be employed to re-evaluate and adapt the value for every iteration.

This paper provides an enhancement in the application of the BoostConv method that can be easily integrated into existing implementations. It provides a means of improving the convergence rates experienced with the original BoostConv method. It also improves the method's robustness in finding steady-state turbulent solutions of the RANS equations, opening up the modified BoostConv method to be used to use in a wide variety of industrial flow cases.

CRediT authorship contribution statement

Antariksh Dicholkar: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing, Validation, Visualization. **Frederik Zahle:** Conceptualization, Writing – review & editing, Supervision, Software, Project administration. **Niels N. Sørensen:** Conceptualization, Writing – review & editing, Supervision, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Pre-simulation studies

A.1. Mesh convergence study

The mesh dependence of the standard EllipSys flow solver and the EllipSys flow solver coupled to the modified BoostConv method, also known as the boosted EllipSys solver, is studied. For this purpose we produce an even finer mesh than the L1 mesh with its 384 cells distributed along the object surface and 192 cells in the direction normal to the surface. The finest mesh, called the L0 mesh, has 768 cells distributed along the object surface and 384 cells in the direction normal to the surface. The L0 mesh has been produced primarily for the purpose of the mesh convergence study and is not used in any of the performed simulations.

We compute the lift and drag coefficients at the three mesh levels, namely L0, L1 and L2. The number of cells corresponding to each mesh level can be seen in Table 6. The coarse L2 mesh has 18432 cells, the fine L1 mesh has 73728 cells and the even finer L0 mesh has 294912 cells. For brevity the mesh dependence study for only one of the three cross-sectional geometries, namely the NACA 63(3)-418 airfoil, is shown. The flow over the airfoil is simulated at a Reynolds number of 1×10^7 at an angle of attack of 120° . At these conditions, the flow is expected to be highly separated and results in the non-convergence of flow residuals for the standard EllipSys flow solver, with the residuals displaying limit cycle oscillations. However, with a basis size of 60 and the modified BoostConv relaxation values of 0.1, the boosted EllipSys flow solver converges the solution to near machine precision.

For the standard EllipSys flow solver, the lift and drag coefficients are computed by averaging the values over the number of iterations for which the solution undergoes the limit cycle. Whereas the lift and drag coefficients are computed from the converged flow solution for the boosted EllipSys solver. The lift coefficient (Cl) and drag coefficient (Cd) for the standard and boosted solvers at the three mesh levels L0, L1 and L2 are shown in Table 6.

The convergence of the lift and drag coefficients are quantified by computing the relative errors at each mesh level. The errors are computed by taking the Richardson extrapolations of the lift and drag coefficients as the reference values. The Richardson extrapolation (Richardson and Gaunt, 1927), is computed by assuming that the discrete solution for a given grid spacing has a series representation involving the sum of the exact or continuum solution and a series of discretization independent functions multiplied by increasing powers of the grid spacing. For a second order method, the continuum solution can be approximated by taking the discrete solutions at two different grid spacing (Roache, 2002). The continuum solution for a second order method is then given by

$$f_{\text{exact}} \approx f_{\text{fine}} + \frac{f_{\text{fine}} - f_{\text{coarse}}}{r^2 - 1},$$

$$r = \frac{h_{\text{coarse}}}{h_{\text{fine}}} \quad (20)$$

where f_{exact} is the continuum solution at an infinitely fine mesh, f_{fine} is the discrete solution at the fine grid level, f_{coarse} is the discrete solution at the coarse grid level, h_{coarse} is the coarse grid spacing, h_{fine} is the fine grid spacing and the ratio r is the grid refinement ratio. For the present study, the solution at mesh level L0 is considered as the fine solution or f_{fine} , the solution at mesh level L1 is considered as the coarse solution or f_{coarse} and the grid refinement ratio $r = 2$. The EllipSys flow solver has been shown to be second order (Sørensen, 2009) and thus Eq. (20) can be applied.

The error for the computed lift and drag coefficients, shown in Table 6 is seen to decrease with increasing fineness of the mesh for both the standard and boosted EllipSys solvers. The error at mesh level L2 is too large to be acceptable for both the solvers and as a consequence any solutions at this mesh level can be safely regarded as not grid converged. The relative errors in the lift and drag coefficients at L2 are comparable between their respective values for the standard and boosted solvers. In contrast, the errors at mesh level L1 are much lower than at mesh level L2 for both the standard and boosted solvers. Furthermore, at mesh level L2 the errors in lift and drag coefficients for the standard solver are almost twice of their respective values for the boosted solver. This suggests that the forces obtained from the converged solution of the boosted solver is attaining mesh independence at a faster rate than the averaged forces from the limit cycle solution of the standard solver. The errors in both the standard and boosted solvers decrease even more at mesh level L0. At L0, the errors are once again lower in the boosted solver than in the standard solver, but are nonetheless comparable and low enough for the solution to be considered converged in both cases.

The convergence of the lift and drag coefficient values with their respective Richardson extrapolations for both the standard and boosted EllipSys solvers are shown in Figs. 11(a) and 11(b). It can be assessed both visually and quantitatively that the error reduction in the lift and drag coefficient values between mesh levels L1 and L0 are less than 5% for both the standard and boosted solvers. Such a low reduction in the errors cannot offset the high computational cost incurred due to the quadrupling of the number of cells from L1 to L0. The L1 mesh presents a fair compromise between computational cost and mesh independence. Thus, the L1 mesh is chosen for the simulations carried out in this study.

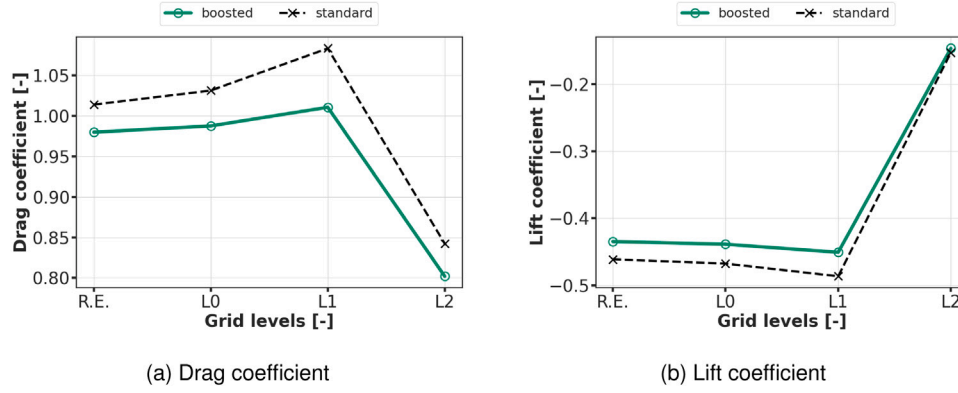


Fig. 11. Lift and drag coefficients for the meshes L0, L1 and L2 and the extrapolated continuum values computed using Richardson extrapolation. The lift and drag coefficients are computed for flow over a 2-D cross-section of the NACA 63(3)-418 airfoil at an angle of attack of 120° .

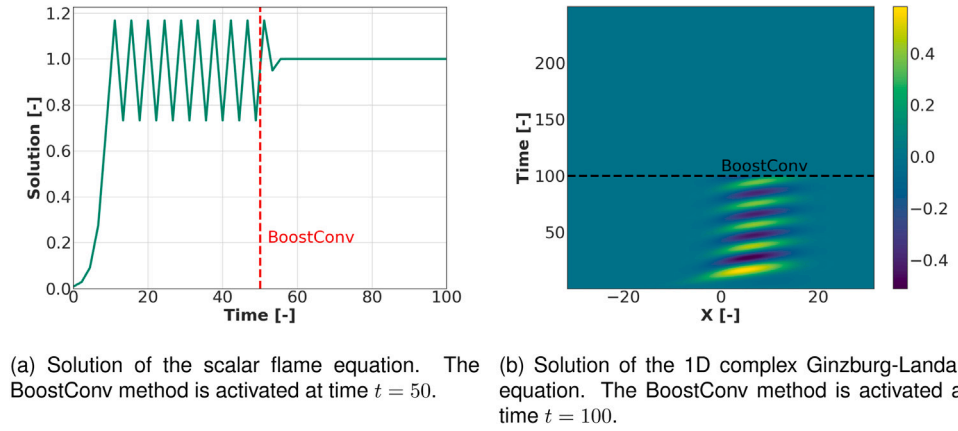


Fig. 12. Computing steady-state of limit cycle oscillations (LCO) with the BoostConv method for the stiff flame equation and the 1D complex Ginzburg-Landau equation.

Table 6

Mesh convergence study for the standard EllipSys flow solver and the boosted EllipSys flow solver. The lift coefficient (Cl) and the drag coefficient (Cd) are computed for flow over a 2-D cross-section of the NACA 63(3)-418 airfoil at an angle of attack of 120° and for a Reynolds number of 1×10^7 . The error is computed by using the values from the Richardson extrapolations as the reference values.

Mesh	Cells	Standard EllipSys				Boosted EllipSys			
		Cl [-]	Error [%]	Cd [-]	Error [%]	Cl [-]	Error [%]	Cd [-]	Error [%]
L2	18432	-0.1529	66.84	0.8423	16.92	-0.1467	66.26	0.8020	18.14
L1	73728	-0.4865	5.49	1.0832	6.84	-0.4509	3.70	1.0105	3.14
L0	294912	-0.4676	1.37	1.0312	1.71	-0.4388	0.93	0.9874	0.78
Extrapolation	∞	-0.4612	0.	1.014	0.	-0.4348	0.	0.9798	0.

A.2. Verification

For verifying the implementation of BoostConv, we use a stiff numerical ordinary differential equation from Moler (2008) and a convection-diffusion PDE with a non-linear source term from Citro et al. (2017a) as test cases. The scalar test case is chosen as the flame equation shown below,

$$y'(t) = y(t)^2 - y(t)^3, \quad y(0) = \delta, \quad 0 \leq t \leq 2/\delta \quad (21)$$

where $y(t)$ is the non-dimensional radius of the flame ball, t represents the non-dimensional time and δ is the initial non-dimensional radius of the flame ball which is a small value. Physically, the model represents the growth of a ball of flame. It is being solved for $\delta = 0.009$ and is integrated in time with an explicit forward Euler scheme discretized with 100 time-steps in the time domain. Eq. (21) is numerically stiff and an explicit solver is not able to converge it to the steady-state. This is evident from Fig. 12(a) showing the evolution of the solution with

time. The solution displays LCO around its analytical steady-state value of $y = 1$. This is seen until a time $t = 50$ when the BoostConv method is activated. For a basis size of 2 the solution converges to its exact steady-state value. The complex Ginzburg-Landau (CGL) equation is selected as the one-dimensional test case and is shown below,

$$\frac{\partial q}{\partial t} + v \frac{\partial q}{\partial x} - \gamma \frac{\partial^2 q}{\partial x^2} - \mu(x)q + |q|^2 q = 0 \quad (22)$$

where q is the complex amplitude, t is time, x is a spatial dimension, v is the complex convection coefficient, γ is the complex diffusion coefficient and $\mu(x)$ is a quadratic function. The explanation of this equation and the terms can be found in Ilak et al. (2010). The constants have the same values as that in Citro et al. (2017a). It is a convection-diffusion type equation and has been chosen for its use in modeling vortex shedding in bluff body wakes, making it highly relevant for the present study. The diffusion term $\partial^2 q / \partial x^2$ and the convection term $\partial q / \partial x$ are discretized in space using central difference and first order backward difference schemes, respectively. The equation is marched

in time using an explicit forward Euler scheme with a discrete time-step of $\Delta t = 0.001$. The spatial domain $[-10\pi, 10\pi]$ is discretized with a uniform spacing consisting of 100 spatial elements. Furthermore, Dirichlet boundary conditions $q_{lb} = 0$ and $q_{rb} = 0$ are enforced at the left and right boundaries respectively and the initial value of the solution at time $t = 0$ is taken as a complex sine wave $\sin(x)e^{ix}$. As in Citro et al. (2017a), the values of the constants are chosen as $\gamma = 1 - i$, $\mu_0 = 0.52$, $\mu_2 = -0.01$ and $\nu = 2 + 0.2i$. This results in an unstable region in the spatial range of $-10.1 < x < 10.1$ leading to the development of limit-cycle oscillations in the solution about its steady-state analytical value of $q = 0$. This is seen until time $t = 100$ in Fig. 12(b) showing the evolution of the solution with time. At time $t = 100$ the BoostConv method is activated with a basis dimension of 15 and the solution is seen to converge to machine precision.

References

- Åkervik, Espen, Brandt, Luca, Henningson, Dan S., Hoepffner, Jérôme, Marxen, Olaf, Schlatter, Philipp, 2006. Steady solutions of the Navier-Stokes equations by selective frequency damping. *Phys. Fluids* (ISSN: 10706631) 18 (6), 6–9. <http://dx.doi.org/10.1063/1.2211705>.
- Bechmann, Andreas, Sørensen, Niels N., Zahle, Frederik, 2011. CFD simulations of the MEXICO rotor. *Wind Energy* 14 (5), 677–689.
- Bertagnolio, F., Sørensen, Niels N., Johansen, Jeppe, Fuglsang, P., 2001. Wind turbine airfoil catalogue. (ISSN: 01062840) ISBN: 8755029108.
- des Bosc, Pierre-Emmanuel, Kuhlmann, Hendrik C., 2019. Three-dimensional flow in a shear-driven cube. *Proc. Appl. Math. Mech.* (ISSN: 1617-7061) 19 (1), 1–2. <http://dx.doi.org/10.1002/pamm.201900090>.
- Boujo, E., Fani, A., Gallaire, F., 2019. Second-order sensitivity in the cylinder wake: Optimal spanwise-periodic wall actuation and wall deformation. *Phys. Rev. Fluids* (ISSN: 246990X) 4 (5), 53901. <http://dx.doi.org/10.1103/PhysRevFluids.4.053901>, arXiv:1808.02796.
- Boyd, Stephen, Boyd, Stephen P., Vandenberghe, Lieven, 2004. *Convex Optimization*. Cambridge University Press.
- Bucci, Michele Alessandro, 2018. Subcritical and supercritical dynamics of incompressible flow over miniaturized roughness elements (Ph.D. thesis). Arts et Métiers ParisTech.
- Chisholm, Todd T., Zingg, David W., 2009. A Jacobian-free Newton-Krylov algorithm for compressible turbulent fluid flows. *J. Comput. Phys.* (ISSN: 10902716) 228 (9), 3490–3507. <http://dx.doi.org/10.1016/j.jcp.2009.02.004>.
- Citro, Vincenzo, 2019. Simple and efficient acceleration of existing multigrid algorithms. *AIAA J.* (ISSN: 00011452) 57 (6), 2244–2247. <http://dx.doi.org/10.2514/1.J058248>.
- Citro, V., Giannetti, F., Luchini, P., Auteri, F., 2015a. Global stability and sensitivity analysis of boundary-layer flows past a hemispherical roughness element. *Phys. Fluids* (ISSN: 10897666) 27 (8), <http://dx.doi.org/10.1063/1.4928533>.
- Citro, V., Luchini, P., Giannetti, F., Auteri, Franco, 2015b. Boundary-layer flows past an hemispherical roughness element: DNS, global stability and sensitivity analysis. *Procedia IUTAM* 14, 173–181.
- Citro, V., Luchini, P., Giannetti, F., Auteri, F., 2017a. Efficient stabilization and acceleration of numerical simulation of fluid flows by residual recombination. *J. Comput. Phys.* 344, 234–246. <http://dx.doi.org/10.1016/j.jcp.2017.04.081>, ISSN: 10902716, 00219991.
- Citro, Vincenzo, Siconolfi, Lorenzo, Fabre, David, Giannetti, Flavio, Luchini, Paolo, 2017b. Stability and sensitivity analysis of the secondary instability in the sphere wake. *AIAA J.* (ISSN: 00011452) 55 (11), 3661–3668. <http://dx.doi.org/10.2514/1.J055376>.
- Citro, V., Tchoufag, J., Fabre, D., Giannetti, F., Luchini, P., 2016. Linear stability and weakly nonlinear analysis of the flow past rotating spheres. *J. Fluid Mech.* (ISSN: 14697645) 807, 62–86. <http://dx.doi.org/10.1017/jfm.2016.596>.
- Dhert, Tristan, Ashuri, Turaj, Martins, Joaquim R.R.A., 2017. Aerodynamic shape optimization of wind turbine blades using a Reynolds-averaged Navier–Stokes model and an adjoint method. *Wind Energy* 20 (5), 909–926.
- Drela, Mark, 1989. XFoil: An analysis and design system for low Reynolds number airfoils. In: *Low Reynolds Number Aerodynamics*. Springer, pp. 1–12.
- Fischer, Paul F., Lottes, James W., Kerkemeier, Stefan G., 2008. nek5000 Web page. URL <http://nek5000.mcs.anl.gov>.
- Gfortran team, 2021. GNU Fortran Wiki. URL <https://gcc.gnu.org/wiki/GFortran>.
- Görtz, Stefan, Möller, Joakim, 2004. Evaluation of the recursive projection method for efficient unsteady turbulent CFD simulations. In: 24th International Congress of the Aeronautical Sciences. pp. 1–13. URL http://www.icas.org/ICAS_ARCHIVE/ICAS2004/PAPERS/032.PDF%5Cnpapers://bf0b7ec0-ce34-4fd4-bd01-c7472aa2d712/Paper/p2417.
- He, Xiaolong, Li, Jichao, Mader, Charles A., Yildirim, Anil, Martins, Joaquim R.R.A., 2019. Robust aerodynamic shape optimization—From a circle to an airfoil. *Aerosp. Sci. Technol.* (ISSN: 12709638) 87, 48–61. <http://dx.doi.org/10.1016/j.ast.2019.01.051>.
- Heil, Matthias, Hazel, Andrew L., Boyle, Jonathan, 2008. Solvers for large-displacement fluid-structure interaction problems: Segregated versus monolithic approaches. *Comput. Mech.* (ISSN: 01787675) 43 (1), 91–101. <http://dx.doi.org/10.1007/s00466-008-0270-6>.
- Hicken, Jason E., Zingg, David W., 2008. Parallel Newton-Krylov solver for the euler equations discretized using simultaneous-approximation terms. *AIAA J.* (ISSN: 00011452) 46 (11), 2773–2786. <http://dx.doi.org/10.2514/1.34810>.
- Ilak, Miloš, Bagheri, Shervin, Brandt, Luca, Rowley, Clarence W., Henningson, Dan S., 2010. Model reduction of the nonlinear complex Ginzburg-Landau equation. *SIAM J. Appl. Dyn. Syst.* (ISSN: 15360040) 9 (4), 1284–1302. <http://dx.doi.org/10.1137/100787350>.
- Johansen, Jeppe, Sørensen, Niels N., Michelsen, JA, Schreck, S., 2002. Detached-eddy simulation of flow around the NREL phase VI blade. *Wind Energy* 5 (2–3), 185–197.
- Kolmogorov, Dmitry K., Shen, Wen Z., Sørensen, Niels N., Sørensen, Jens N., 2015. Fully consistent SIMPLE-like algorithms on collocated grids. *Numer. Heat Transfer B* (ISSN: 15210626) 67 (2), 101–123. <http://dx.doi.org/10.1080/10407790.2014.949583>.
- Lasher, William C., Sonnenmeier, James R., 2008. An analysis of practical RANS simulations for spinnaker aerodynamics. *J. Wind Eng. Ind. Aerodyn.* 96 (2), 143–165.
- Lemke, Mathias, Citro, Vincenzo, Giannetti, Flavio, 2021. External acoustic control of the laminar vortex shedding past a bluff body. *Fluid Dyn. Res.* (ISSN: 0169-5983) 53 (1), 015506. <http://dx.doi.org/10.1088/1873-7005/abd8dd>.
- Leonard, B.P., 1979. A stable and accurate convective modelling procedure based on quadratic upstream interpolation. *Comput. Methods Appl. Mech. Engrg.* 19 (1), 59–98. [http://dx.doi.org/10.1016/0045-7825\(79\)90034-3](http://dx.doi.org/10.1016/0045-7825(79)90034-3), ISSN: 18792138, 00457825.
- Lupi, Valerio, Canton, Jacopo, Schlatter, Philipp, 2020. Global stability analysis of a 90°-bend pipe flow. *Int. J. Heat Fluid Flow* 86, 108742.
- Madsen, Mads H. Aa, Zahle, Frederik, Sørensen, Niels N., Martins, Joaquim R.R.A., 2019. Multipoint high-fidelity CFD-based aerodynamic shape optimization of a 10 MW wind turbine. *Wind Energy Sci.* 4 (2), 163–192.
- Menter, Florian R., 1993. Zonal two equation kappa-omega turbulence models for aerodynamic flows.
- Michelsen, J.A., 1992. Basis3D - A Platform for Development of Multiblock PDE Solvers. Tech. rep, Risø National Laboratory.
- Michelsen, J.A., 1994. Block Structured Multigrid Solution of 2D and 3D Elliptic PDE'S. Tech. rep, Technical University of Denmark.
- Moler, Cleve B., 2008. *Numerical Computing with MATLAB: Revised Reprint, Vol. 87*. Siam.
- Negi, P.S., Hanifi, A., Henningson, D.S., 2020. On the linear global stability analysis of rigid-body motion fluid-structure-interaction problems. *J. Fluid Mech.* (ISSN: 14697645) 903, <http://dx.doi.org/10.1017/jfm.2020.685>.
- Osusky, Michal, Zingg, David W., 2013. Parallel Newton-Krylov-Schur flow solver for the Navier-Stokes equations. *AIAA J.* (ISSN: 00011452) 51 (12), 2833–2851. <http://dx.doi.org/10.2514/1.J052487>.
- Patankar, Suhas V., 1981. A calculation procedure for two-dimensional elliptic situations. *Numer. Heat Transfer* 4 (4), 409–425. <http://dx.doi.org/10.1080/01495728108961801>, ISSN: 23314168, 01495720.
- Patankar, S.V., Spalding, D.B., 1972. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *Int. J. Heat Mass Transfer* 15 (10), 1787–1806. [http://dx.doi.org/10.1016/0017-9310\(72\)90054-3](http://dx.doi.org/10.1016/0017-9310(72)90054-3), ISSN: 00179310, 18792189.
- Rhie, C.M., Chow, W.L., 1983. Numerical study of the turbulent flow past an airfoil with trailing edge separation. *AIAA J.* (ISSN: 00011452) 21 (11), 1525–1532. <http://dx.doi.org/10.2514/3.8284>.
- Richardson, Lewis Fry, Gaunt, J. Arthur, 1927. The deferred approach to the limit. *Philos. Trans. R. Soc. Lond. Ser. A Math. Phys. Eng. Sci.* 226 (636–646), 299–361.
- Richez, F., Leguille, M., Marquet, O., 2016. Selective frequency damping method for steady RANS solutions of turbulent separated flows around an airfoil at stall. *Comput. & Fluids* (ISSN: 00457930) 132 (June), 51–61. <http://dx.doi.org/10.1016/j.compfluid.2016.03.027>.
- Roache, P.J., 2002. Quantification of uncertainty in computational fluid dynamics. *Annu. Rev. Fluid Mech.* (ISSN: 0066-4189) 29 (1), 123–160. <http://dx.doi.org/10.1146/annurev.fluid.29.1.123>.
- Saglietti, Clio, Schlatter, Philipp, Wadbro, Eddie, Berggren, Martin, Henningson, Dan S., 2018a. Topology optimization of heat sinks in a square differentially heated cavity. *Int. J. Heat Fluid Flow* (ISSN: 0142727X) 74 (September), 36–52. <http://dx.doi.org/10.1016/j.ijheatfluidflow.2018.08.004>.
- Saglietti, Clio, Schlatter, Philipp, Wadbro, Eddie, Berggren, Martin, Henningson, Dan S., 2018b. Topology optimization of heat sinks in a square differentially heated cavity. *Int. J. Heat Fluid Flow* (ISSN: 0142727X) 74 (March), 36–52. <http://dx.doi.org/10.1016/j.ijheatfluidflow.2018.08.004>.
- Saglietti, Clio, Wadbro, Eddie, Berggren, Martin, Henningson, D.A.N.S., 2018c. Heat transfer maximization in a three dimensional conductive differentially heated cavity by means of topology optimization. In: *Proceedings of the Seventh European Conference on Computational Fluid Dynamics. ECCM-E CFD*, June. pp. 1–12.

- Shroff, Gautam M., Keller, Herbert B., 1993. Stabilization of unstable procedures: The recursive projection method. *SIAM J. Numer. Anal.* 30 (4), 1099–1120. <http://dx.doi.org/10.1137/0730057>, ISSN: 10957170, 00361429.
- Sierra, J., Fabre, D., Citro, V., Giannetti, F., 2020. Bifurcation scenario in the two-dimensional laminar flow past a rotating cylinder. *J. Fluid Mech.* (ISSN: 14697645) 905, <http://dx.doi.org/10.1017/jfm.2020.692>.
- Singh, S.P., Mittal, S., 2005. Flow past a cylinder: Shear layer instability and drag crisis. *Internat. J. Numer. Methods Fluids* 47 (1), 75–98.
- Sørensen, Niels N., 1995. General Purpose Flow Solver Applied to Flow Over Hills (Ph.D. thesis). Technical University of Denmark.
- Sørensen, Niels N., 1998. HypGrid2D a 2-D Mesh Generator. Risø National Laboratory Roskilde, Denmark.
- Sørensen, Niels N., 2009. CFD modelling of laminar-turbulent transition for airfoils and rotors using the γ - Re_{θ} model. *Wind Energy* (ISSN: 10954244) 12 (8), 715–733. <http://dx.doi.org/10.1002/we.325>.
- Sorensen, N., Michelsen, Jess, 2000. Aerodynamic predictions for the unsteady aerodynamics experiment phase-II rotor at the National Renewable Energy Laboratory. In: 2000 ASME Wind Energy Symposium. pp. 37.
- Sørensen, Niels N., Michelsen, J.A., Schreck, S., 2002. Navier-Stokes predictions of the NREL phase VI rotor in the NASA ames 80 ft $\times$ 120 ft wind tunnel. *Wind Energy* 5 (2–3), 151–169. <http://dx.doi.org/10.1002/we.64>, ISSN: 10991824, 10954244.
- Sørensen, Niels N., Schreck, Scott, 2014. Transitional DDES computations of the NREL phase-VI rotor in axial flow conditions. *J. Phys. Conf. Ser.* 555 (1), 012096. <http://dx.doi.org/10.1088/1742-6596/555/1/012096>, ISSN: 17426596, 17426588.
- Sørensen, Niels N., Zahle, Frederik, Boorsma, K., Schepers, G., 2016. CFD computations of the second round of MEXICO rotor measurements. *J. Phys. Conf. Ser.* 753, 022054.
- Timmer, W.A., Bak, C., 2013. Aerodynamic characteristics of wind turbine blade airfoils. In: *Advances in Wind Turbine Blade Design and Materials*. Woodhead Publishing Limited, ISBN: 9780857094261, pp. 109–149. <http://dx.doi.org/10.1533/9780857097286.1.109>.
- Van Rossum, Guido, Drake, Fred L., 2009. Python 3 Reference Manual. CreateSpace, Scotts Valley, CA, ISBN: 1441412697.
- Veers, Paul, Dykes, Katherine, Lantz, Eric, Barth, Stephan, Bottasso, Carlo L., Carlson, Ola, Clifton, Andrew, Green, John, Green, Peter, Holttinen, Hannele, et al., 2019. Grand challenges in the science of wind energy. *Science* 366 (6464).
- Xu, Shenren, Radford, David, Meyer, Marcus, Müller, Jens Dominik, 2015. Stabilisation of discrete steady adjoint solvers. *J. Comput. Phys.* (ISSN: 10902716) 299, 175–195. <http://dx.doi.org/10.1016/j.jcp.2015.06.036>.
- Yildirim, Anil, Kenway, Gaetan K.W., Mader, Charles A., Martins, Joaquim R.R.A., 2019. A Jacobian-free approximate Newton–Krylov startup strategy for RANS simulations. *J. Comput. Phys.* (ISSN: 00219991) (June), <http://dx.doi.org/10.1016/j.jcp.2019.06.018>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0021999119304255>.